

Toward Scalable, Cost-Efficient, and Fault-Tolerant Federated Learning Systems

Abstract

(xxx₁: add abstract, please...)

1 Introduction

(xxx₂: We need many references.)

Background. Federated Learning (FL) enables collaborative machine learning across distributed data sources without centralizing raw data [11, 17]. By keeping sensitive data on client devices and sharing only model updates, FL addresses critical privacy requirements in healthcare, finance, and mobile computing. Production FL deployments now serve millions of devices. For example, Google’s Gboard keyboard trains on mobile devices worldwide, while hospitals explore FL for collaborative medical research without exposing patient records [5, 23].

Despite FL’s promise and early deployments, *a significant gap exists between research prototypes and systems that can operate reliably at scale*. While academic FL frameworks/systems (xxx₃: ref FL papers) demonstrate the feasibility of distributed training, they fall short when deployed at scale: managing thousands of unreliable clients, operating within resource constraints, and handling infrastructure failures (xxx₄: totoro?) Totoro [4].

Limitations of Current FL Systems. We identify three critical gaps between existing FL frameworks and production requirements:

(1) Architectural scalability limits. Current FL systems coordinate all clients through a single aggregator, creating a fundamental bottleneck. As client count grows, the aggregator must process increasing numbers of model updates while maintaining responsive communication with all participants. This centralized design fails to scale beyond thousands of clients. While deploying multiple aggregators could address this limitation, it requires proportionally more infrastructure resources and complex coordination mechanisms.

(2) Expensive operational costs. For large-scale deployment, FL systems have to rely on expensive on-demand cloud instances to ensure availability. As deployments scale to support more clients, infrastructure costs increase proportionally,

requiring more aggregators, greater network bandwidth, and additional storage. Yet these expensive resources remain underutilized during off-peak (or idle) hours when client participation drops. This linear cost scaling with deployment size makes large-scale FL financially infeasible for most organizations.

(3) Insufficient fault tolerance. Spot instances offer an economical alternative to expensive on-demand resources, providing 70 - 90% cost savings. However, current FL frameworks lack the fault tolerance capabilities needed to leverage these cost-effective resources. Production systems already face routine failures—server crashes, network partitions, and client disconnections [6–8, 21]. When using spot instances that can be terminated with just 2-minute warnings, existing frameworks *may* lose entire training rounds and waste substantial computation. This inability to handle preemptible infrastructure prevents FL systems from achieving the cost efficiency needed for large-scale deployment.

Research Challenges. These limitations motivate us to tackle three foundational research challenges (RCs):

RC #1: HOW DO WE BUILD FL SYSTEMS THAT SCALE HORIZONTALLY? Distributing FL across multiple aggregators requires careful coordination to maintain training correctness. We need architectures that automatically balance load while ensuring model consistency, without introducing excessive synchronization overhead that would negate scaling benefits.

RC #2: HOW DO WE MAKE FL ECONOMICALLY VIABLE? We must exploit cheap but unreliable resources like spot instances while maintaining training stability. This requires co-designing scaling policies and fault tolerance mechanisms that understand FL’s unique workload patterns and consistency requirements.

RC #3: HOW DO WE ENSURE TRAINING PROGRESS DESPITE (INFRASTRUCTURE- OR RESOURCE-LEVEL) FAILURES? FL systems need efficient checkpointing, fast failure detection, and seamless recovery mechanisms. When failures occur, the system should gracefully redistribute work without disrupting active clients or losing significant progress.

Research Objectives. To address these research challenges, we aim to design and implement a FL system that overcomes the fundamental limitations of current approaches. We pursue

three research objectives (ROs):

RO #1: We build an FL system that scales horizontally to support large numbers of clients without creating new bottlenecks (*e.g.*, degraded training efficiency). Our design distributes aggregation load while maintaining consistency guarantees required for correct training.

RO #2: We reduce FL deployment costs by an order of magnitude through intelligent resource management. By utilizing spot instances and implementing demand-aware scaling, we make large-scale FL economically feasible.

RO #3: We ensure continuous training progress despite infrastructure failures (*e.g.*, spot failure/revocation). Our fault tolerance mechanisms treat failures as normal events, maintaining training momentum through efficient checkpointing, state management, and graceful degradation.

These objectives are interdependent. Our system co-designs scalability, cost efficiency, and fault tolerance to create a practical FL solution.

2 Understanding FL System Challenges

We first analyze the technical challenges in building scalable and reliable FL systems. We examine why existing approaches fail at scale and identify specific system-level problems that must be addressed.

2.1 The Aggregation Bottleneck in FL Systems

2.1.1 The Aggregation Bottleneck

In FL systems, the aggregator serves as the central coordinator that manages client participation, collects model updates, and computes global models through aggregation. Single-aggregator designs require this one node to handle all tasks: tracking model versions for every client, maintaining client metadata, processing incoming updates, and performing aggregation computations within tight time windows. As client populations grow, these concurrent responsibilities create severe bottlenecks. For example, connection management overhead increases significantly, memory usage scales with client count, and CPU becomes saturated during aggregation phases. These compounding resource constraints fundamentally limit the system’s ability to scale beyond thousands of clients.

The stateful nature of FL workloads makes horizontal scaling particularly challenging. Unlike stateless web services where requests can be freely distributed, FL requires careful coordination to maintain training correctness and ensure model convergence.

2.1.2 Distributed/Scalable Aggregation Challenges

Employing multiple aggregators appears to be a straightforward solution to scaling limitations, but this approach intro-

duces significant complexity that existing FL frameworks do not address:

(1) Load balancing across aggregators. With multiple aggregators, clients must be assigned to specific aggregators for each training round. However, client availability in FL is highly dynamic. *i.e.*, mobile devices connect and disconnect based on network conditions, battery status, and user behavior [3, 12, 13]. Moreover, device mobility can require clients to switch between geographically distributed aggregators to maintain low latency connections [15]. Static assignment strategies that permanently map clients to aggregators quickly lead to load imbalance, with some aggregators overwhelmed while others sit idle.

(2) Model version consistency. With multiple aggregators, different nodes may operate on different versions of the global model due to asynchronous client updates and network delays. For example, while one aggregator processes updates based on model version N , another might still be working with version $N-1$. If not properly synchronized, these version mismatches can cause the global model to diverge or fail to converge, defeating the purpose of collaborative learning. This requires distributed consensus on model versioning while minimizing synchronization overhead.

With multiple aggregators, model parameters, aggregation buffers, and training metadata must be coordinated across aggregators. Naive approaches that broadcast full model states between aggregators negate the benefits of distribution. However, as already identified, communication overhead can be the main bottleneck for FL at scale (cite FedAvg, FedProx paper here). Therefore, We need selective synchronization protocols that maintain consistency while allowing meaningful parallelism.

(3) State synchronization overhead. With multiple aggregators, model parameters, aggregation buffers, and training metadata must be coordinated across aggregators. Naive approaches that broadcast full model states between aggregators negate the benefits of distribution. This is particularly problematic in FL, where communication overhead is already identified as a primary bottleneck [14, 17]. Therefore, efficient multi-aggregator systems require selective synchron

2.2 The Cost Challenge of Scaling FL

2.2.1 The Economics Problem

FL systems typically deploy on cloud infrastructure using on-demand instances to ensure availability. However, FL workloads exhibit highly variable client participation, from thousands during peak hours to hundreds during off-peak [3, 12, 24]. Even with autoscaling, deploying FL systems at scale requires substantial infrastructure that incurs high costs. On-demand instances charge by the hour regardless of actual utilization, and FL’s synchronized training rounds prevent

fine-grained scaling that could reduce costs. This makes large-scale FL deployments expensive, even with standard cloud elasticity.

2.2.2 The Promise and Peril of Spot Instances

Spot instances offer an attractive solution with 70 – 90% cost savings compared to on-demand instances. For FL workloads that can tolerate some disruption, spot instances could dramatically reduce operational costs. However, they introduce a critical challenge: instances can be terminated with minimal warning (typically 2 minutes) at any time. Even worse, when cloud providers need capacity back, they often reclaim many spot instances at once, leading to a situation where a system could lose a significant portion of its resources in just a few minutes.

To maximize cost-benefit from spot instances, FL systems need fundamental architectural changes, such as supporting frequent checkpointing, proactive workload migration. In short, spot instances can only save money if the FL system can handle frequent failures, but most existing FL frameworks cannot.

2.3 The Fault Tolerance Challenge

Using spot instances for cost-efficient FL makes fault tolerance essential, as unexpected failures (or VM revocations) are common. Several technical challenges must be addressed:

(xxx5: these need to be updated/changed for consistency with the later sections.)

First, **efficient checkpointing becomes critical**. The FL system must persist global model states, partially aggregated updates, and versioning metadata frequently enough to minimize loss when instances are preempted. However, excessive checkpointing creates overhead that reduces training efficiency. Finding the optimal checkpoint frequency requires balancing preemption probability against checkpoint cost, which is a non-trivial optimization problem unique to spot-based FL deployments.

Second, **the system needs rapid and proactive state migration capabilities**. Rather than waiting for termination notices, the system should predict likely preemptions based on spot price trends, instance runtime patterns, and cloud provider behaviors. When preemption risk increases or when the two-minute termination notice arrives, the system must quickly transfer state to surviving instances. This demands compressed state representations to minimize transfer time, pre-established migration targets to avoid coordination overhead during crisis, and graceful client handoff protocols that maintain training continuity.

Finally, **preventing cascading failures is essential**. When multiple instances terminate simultaneously, the remaining

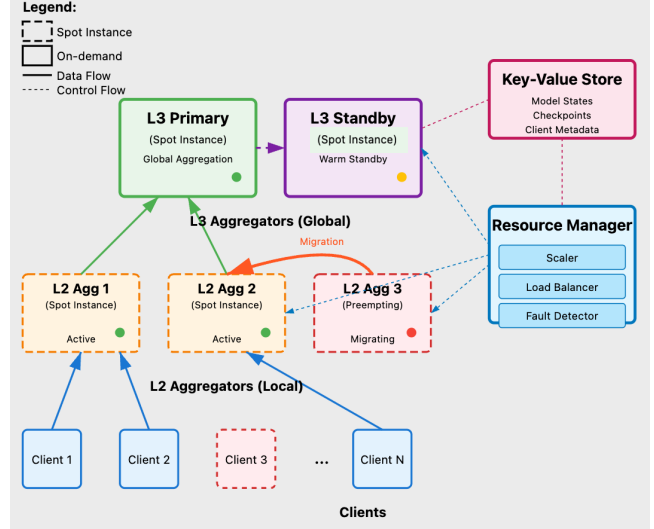


Figure 1: **example fig. Proposed Hierarchical and Scalable FL Architecture** The proposed system consists of three main layers: (1) ‘Clients at the edge’ that perform local training, (2) ‘L2 aggs (local aggs) deployed on spot instances’ that collect updates from assigned clients and compute partial aggregations, and (3) ‘L3 aggs (global aggs) also on spot instances’ that perform final model aggregation. The ‘Resource Manager’ serves as the control plane, coordinating client assignment, dynamic scaling, and fault detection across all components. A ‘distributed key-value store’ provides persistent storage for model states, checkpoints, and client metadata. The diagram shows both normal data flow (solid arrows) for model updates and control flow (dashed arrows) for system orchestration. During spot instance preemptions, the system supports rapid state migration between aggregators (shown as migration path) to maintain training continuity.

infrastructure must process the additional load without becoming overwhelmed. This requires sophisticated admission control to limit new connections, load shedding mechanisms to prioritize critical operations, and degraded operation modes that maintain system stability under reduced capacity.

3 Scalable FL System Design

We now present the design of our scalable/hierarchical FL system, built to address the scalability, cost efficiency, and fault tolerance challenges described in Section 2. We detail how the system integrates a two-tier aggregation hierarchy, an adaptive resource manager, spot instance–optimized deployment, and layered fault tolerance to support large-scale, cost-aware, and resilient FL training.

3.1 System Overview

We design a hierarchical FL system that jointly addresses scalability, cost efficiency, and fault tolerance, as motivated by the challenges in Section 2. Figure 1 shows the architecture, which integrates four core components: a two-tier aggregation hierarchy, a centralized **resource manager**¹, a distributed key-value store, and multi-level fault tolerance mechanisms.

The *two-tier aggregation hierarchy* decouples client coordination from global model computation, with both tiers running on spot instances. Local (L2) aggregators manage client connections and perform partial aggregations, while global (L3) aggregators enforce model consistency and produce the final global model. This separation enables each tier to scale independently based on workload patterns and load fluctuations.

The *resource manager* operates as the control plane, monitoring system state, making scaling decisions, orchestrating workload migration, and coordinating fault recovery. By centralizing control logic while distributing data operations, we achieve both system-wide coordination and scalable performance.

The *key-value store* provides persistent, distributed storage for model states, checkpoints, and metadata. This storage layer enables fast recovery from failures, efficient state migration between aggregators, and consistent model versioning across aggregators.

Together, these components meet our three objectives. The hierarchical architecture distributes aggregation load via *horizontal scalability* (RO #1). Running all components on spot instances with demand-aware resource management enables *cost efficiency* (RO #2). Fault tolerance mechanisms, *e.g.*, proactive migration, multi-level checkpointing, and edge offloading, ensure *reliable operation* despite frequent spot instance preemptions (RO #3).

3.2 Enabling Horizontal Scaling

(IKK₁: This is our RO#1 to address RC#1)

We enable horizontal scaling by decoupling client coordination from global aggregation, allowing each tier to expand without bottlenecks.

3.2.1 Hierarchical Aggregation Architecture

Our two-tier hierarchy splits aggregation between *L2 aggregators* that manage clients and *L3 aggregators* that compute the global model. This separation is motivated by a key observation: client coordination and model aggregation have fundamentally different scaling characteristics and resource demands and thus benefit from independent optimization.

¹ (xxx₆: this may be no longer true due to recent modification of rm scaling...)

L2 (Local Aggregator) nodes. L2 nodes directly interface with clients, each managing a subset of the clients. In each round, they distribute the current global model, collect client updates, and compute partial aggregations to produce sub-global models. Rather than forwarding raw updates upstream, each L2 sends a single sub-global model representing the aggregate contribution of its clients. L2 nodes maintain client metadata, handle disconnections and reconnections, and checkpoint intermediate state for recovery. This localizes client management overhead and reduces upstream communication by up to two orders of magnitude.

L3 (Global Aggregator) Nodes. L3 nodes aggregate sub-global models from all L2 nodes to produce the final global model. They enforce version consistency, track convergence metrics, and coordinate system-wide checkpoints. Multiple L3 nodes run on spot instances with leader election via the key-value store, enabling seamless failover. (xxx₇: this is unclear, and do we need to emphasize/cite lead election like RAFT??) Processing only sub-global models avoids the scalability bottleneck of single-aggregator designs.

Key-Value Store. The distributed key-value store provides persistent storage for model states, checkpoints, and metadata. It records global model versions, aggregator configurations, client-to-aggregator mappings, and system metrics. Aggregators recover state from the store after failures and load the latest configuration during scaling. Its distributed design ensures availability despite spot instance preemptions.

Operational flow. Figure 2 shows the seven phases of a hierarchical FL communication round.

1. **Round initiation** – L3 publishes global model $v(t)$.
2. **Model distribution** – L2 nodes relay the global model to their assigned clients.
3. **Local training** – clients train locally for several epochs.
4. **Gradient upload** – clients send updates to their L2.
5. **Partial aggregation** – each L2 produces a sub-global model.
6. **Global aggregation** – L3 aggregates all sub-global models into $v(t+1)$.
7. **Global model update** – updated model distributed to all clients.

The synchronization point after phase 5 ensures that all L2 partial aggregations complete before L3 aggregation begins, preventing version mismatches. By hierarchically decomposing aggregation and enforcing coordinated synchronization, the architecture preserves FL’s aggregation semantics while scaling to orders of magnitude more clients than single-aggregator systems.

3.2.2 Resource Manager for Dynamic Load Distribution

The resource manager serves as the control plane for our distributed FL system, maintaining a global view of system

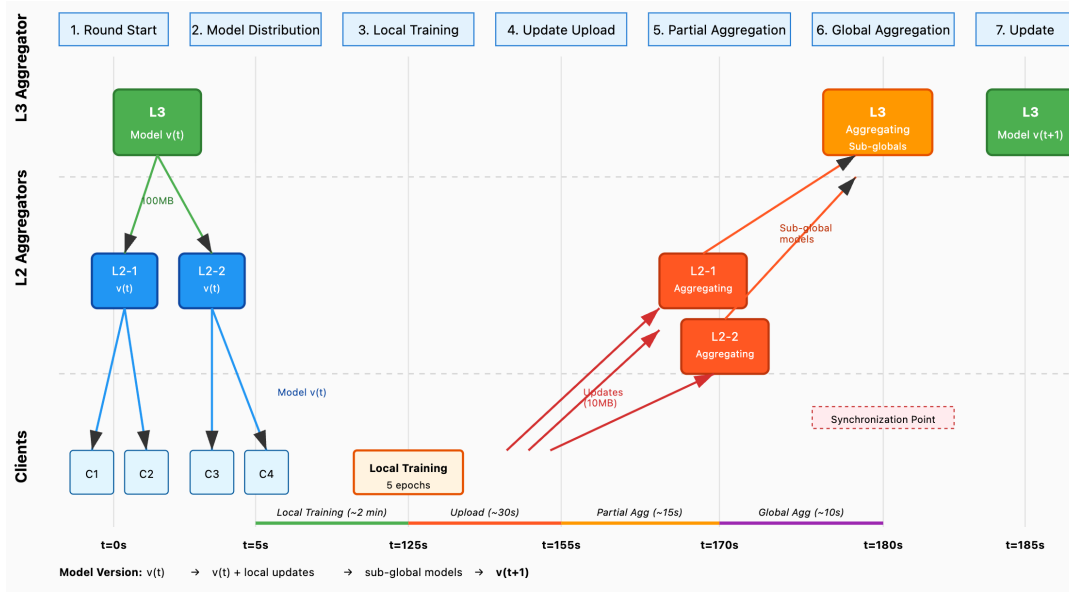


Figure 2: **example fig.** Client-to-aggregator communication flow during one federated learning round. The diagram illustrates the seven phases of hierarchical FL training: (1) ‘Round initiation’ with global model $v(t)$ at L3, (2) ‘Model distribution’ through L2 aggregators to clients, (3) ‘Local training’ on client devices, (4) ‘Gradient upload’ from clients to their assigned L2 aggregators, (5) ‘Partial aggregation’ at L2 aggregators producing sub-global models, (6) ‘Sub-global model aggregation’ at L3, and (7) ‘Global model update’ to $v(t+1)$. The diagram shows typical data sizes (100MB for model distribution, 10MB for gradient uploads) and execution times for each phase. The synchronization point ensures all L2 aggregators complete partial aggregation before L3 begins global aggregation.

state and orchestrating client distribution across aggregators. Unlike hierarchical FL systems with simple and static assignment [2, 10, 15], it applies adaptive placement policies that respond to workload dynamics and changing system conditions.

Client lifecycle management. The resource manager tracks each client from registration to training completion. It assigns clients to appropriate L2 aggregators based on current load, monitors training progress and connection stability, and maintains metadata for informed placement. When clients complete a round or disconnect, their profiles are updated to improve future assignments. This persistent tracking enables accurate system state and optimal client-to-aggregator mapping.

Load balancing. The resource manager uses **threshold-based policies** (xxx8: **change if needed**) to prevent aggregator overload while minimizing migration overhead. Client redistribution is triggered only when imbalance exceeds critical thresholds or when aggregators approach resource limits. Placement decisions consider multiple factors—current client counts, resource utilization, and historical performance trends. By intervening selectively rather than reshuffling continuously, the system maintains stability and avoids single-node bottlenecks. (xxx9: **consider adding pseudo code for load balancing algorithm...**)

Dynamic workload adaptation. FL workloads are highly dynamic, with clients joining, leaving, and reconnecting unpredictably. The resource manager preserves state for temporarily disconnected clients, redistributes work when clients permanently leave, and predicts participation from historical data. Scaling decisions adapt to observed workload trends, maintaining efficiency despite fluctuating client populations. This adaptive approach handles the inherent uncertainty of FL deployments.

Additional Features. (IKK₂: I’m sure that there are more (and important) features as you have been continuously improving the resource manager/spotlight as a whole)

[closing statement.] Through intelligent orchestration of client placement and workload adaptation, the resource manager enables horizontal scaling without sacrificing efficiency, turning distributed client management into a tractable, continuously solved optimization problem.

3.2.3 Maintaining Consistency at Scale

(IKK₃: Sushruth – this is new section to explain model version control and/or consistency. Feel free to change/add/remove...)

Distributed aggregation introduces consistency challenges, which do not exist in single-aggregator systems. When multiple L2 aggregators operate concurrently, they *may* process

updates on different global model versions potentially causing divergence or preventing convergence. Our system implements several mechanisms to maintain training correctness while keeping synchronization overhead low enough to maintain scalability.

Model version control. Each model has a version number that increases with every training round. As shown in Figure 2, when L3 starts a round with global model $v(t)$, the version propagates through L2 aggregators to all clients. Clients train on $v(t)$, L2 aggregators create sub-global models tagged with $v(t)$, and L3 checks that all inputs match $v(t)$ before producing $v(t+1)$. This prevents errors from mixing updates from different rounds. If clients send updates based on old versions, *e.g.*, due to slow training or network delays, the system can either reject them for strict consistency or combine them with reduced weight to preserve part of their contribution.

Efficient synchronization. While strict synchronization ensures correctness, it can severely impact performance in distributed settings. Figure 2 illustrates the critical synchronization point between phases 5 and 6, where all L2 aggregators must complete partial aggregation before L3 begins global aggregation. This barrier ensures version consistency while allowing L2 nodes to operate independently during client interaction phases. Our system employs hierarchical aggregation where L2 nodes send only sub-global models to L3, reducing communication volume by orders of magnitude compared to forwarding raw client updates. We further optimize through delta compression, *e.g.*, transmitting only model differences rather than complete models, which significantly reduces network overhead. The system supports bounded asynchrony, allowing L2 aggregators to begin round $t+1$ while L3 completes aggregation for round t , effectively pipelining computation and communication.

Communication overhead optimization. Network bandwidth often becomes the bottleneck in distributed FL, particularly when model sizes reach hundreds of megabytes. Our system implements gradient compression (*e.g.*, quantization and sparsification techniques) that reduce communication volume while preserving model accuracy. The resource manager schedules model transfers during network idle periods and implements adaptive communication rounds that skip synchronization when model updates are minimal. L2 aggregators cache frequently accessed model components locally, avoiding redundant transfers. These optimizations collectively and substantially reduce network usage compared to naive broadcasting approaches.

[closing statement.] The synchronization protocol ensures mathematical equivalence to centralized federated averaging despite the distributed execution. By carefully managing version consistency, optimizing synchronization points, and minimizing communication overhead, our system achieves the scaling benefits of distribution without sacrificing FL’s convergence properties. This design enables training to pro-

ceed efficiently even as the system scales to thousands of clients across dozens of aggregators.

3.2.4 Elastic Scaling Mechanisms Multi-tier Aggregator

(IKK4: this is for your policy design – feel free to rewrite with new feedback controller with three-level scaler.)

Dynamic scaling matches resource allocation to workload intensity without manual intervention. The resource manager monitors load metrics in real time, adding spot instances when demand spikes and releasing capacity during idle periods to reduce cost.

L2 scaling with hybrid metrics. L2 aggregators face dual pressure from client count and computation load. A pure client-count threshold can miss cases where fewer clients generate heavy computation, while a pure resource threshold can miss growing client queues. Our hybrid scaling policy triggers scale-out when either client count exceeds 1,000 per L2 or resource usage surpasses 80%. Scale-in occurs only when both conditions are met—client count below 300 and resource usage under 30%, to avoid premature termination that could disrupt training. This dual-threshold approach aligns L2 capacity with actual demand rather than relying on a single, potentially misleading metric.

L3 scaling with resource thresholds. L3 aggregators handle a predictable workload determined by the number of L2 nodes. Since each L2 produces one sub-global model per round, L3 load scales with L2 count rather than total clients. We base L3 scaling on aggregation computation time and memory usage, which are more stable than client-facing L2 workloads. As a result, the system typically requires fewer L3 instances, and their count remains stable even as L2 count fluctuates.

FL-aware scaling decisions. Generic autoscaling ignores FL-specific constraints. Our policy considers training round boundaries to avoid disrupting aggregation, predicts next-round load from historical patterns, and provisions resources before demand spikes instead of reacting afterward. Cooldown periods prevent oscillation when load hovers near thresholds. These FL-specific optimizations ensure scaling improves performance without interrupting training progress.

The following pseudocode illustrates our scaling policy implementation: (xxx10: Please add psedocode of scaling policy here...)

Operational behavior. Figure 3 shows system scaling over a 24-hour period. L2 count tracks client participation while maintaining headroom for load spikes. The system scales aggressively during demand increases but conservatively during decreases, prioritizing training stability over minor cost savings. This elastic scaling enables the system to absorb order-of-magnitude variations in client participation without sacrificing throughput.

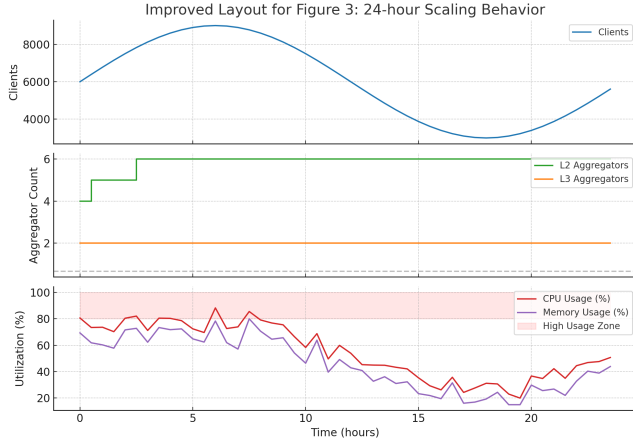


Figure 3: **example fig., specifically this one needs change after discussion w/ Sushruth** 24-hour scaling behavior of the oior scalable FL system. The top panel shows client participation varying from $\approx 1,000$ to 10,000. The middle panel shows L2 and L3 aggregator counts adjusting dynamically to demand, with thresholds and hysteresis preventing oscillations. The bottom panel shows average CPU and memory utilization, with shaded regions indicating high usage zones. Dynamic scaling closely tracks actual demand, maintaining headroom for spikes while reducing resource over-provisioning.

3.3 Cost-Efficient Operation at Scale Through Spot Instances

(IKK₅: This subsection is our RO #2 to address our RC #2.)

We reduce costs by running all L2 and L3 aggregators exclusively on spot instances. Spot instances are up to 70% cheaper than on-demand instances but can be terminated with minimal notice (often within two minutes). (xxx₁₁: add refs.) This aggressive cost strategy is viable only because our fault tolerance mechanisms treat preemptions as routine events rather than failures.

FL workloads align naturally with spot instances. Training proceeds in discrete rounds with built-in checkpoints, allowing state to be saved between rounds without extra overhead. Clients keep their local data and can reconnect to any aggregator after a failure, avoiding complex state reconstruction. The hierarchical architecture adds redundancy: multiple L2 nodes share client load, and multiple L3 nodes maintain synchronized state. When preemptions reduce capacity, the system degrades gracefully by redistributing clients instead of failing entirely.

To enhance resilience, we diversify deployments across instance types and availability zones, lowering the chance of correlated terminations. While individual preemptions cannot be prevented, diversification makes large-scale simultaneous failures unlikely. The resource manager tracks historical pre-

emption patterns to guide placement decisions and refine failure probability estimates.

Our cost optimization goes beyond using spot instances to include proactive resource management. The system scales in rapidly during idle periods, terminating unused instances within minutes to reduce costs. We run with zero on-demand infrastructure, and every component operates on spot instances without any on-demand backup. This approach is viable only because our fault tolerance mechanisms (Section 3.4) sustain reliability despite frequent preemptions. By co-designing cost optimization and fault tolerance, the system aims to achieve substantial cost savings while maintaining reliable training.

3.4 Comprehensive Fault Tolerance

(IKK₆: this section is for RObj #3 to address RC #3.)

Frequent spot preemptions and node failures are inevitable at our scale. We address them with proactive detection, rapid recovery, and efficient workload shifts, detailed in the following subsections.

3.4.1 Proactive Failure Management

We treat spot instance preemptions as the norm rather than the exception. Instead of reacting after terminations, we detect risks early and migrate workloads before failures occur. Our approach combines estimation to predict the likelihood of termination, checkpointing to adjust state saving frequency, and migration to move workloads with minimal disruption.

Spot preemption estimation. We identify termination prone instances before failures occur by tracking historical preemption patterns and instance runtime statistics. The resource manager uses these data to classify such instances, prepare migration plans, and pre select targets in advance. This estimation allows most migrations to complete without relying on last minute alerts. When cloud providers issue termination warnings, they act as a backup mechanism and trigger our pre established migration channels for an immediate and orderly failover, ensuring continuity even if proactive measures have not yet been taken.

Multi-level checkpointing. The checkpointing strategy adjusts the interval based on the estimated preemption risk. Under normal conditions, the system checkpoints at the end of each training round, storing model version numbers and essential metadata. When the risk level is high, the interval shortens to capture additional state and limit progress loss in case of failure. Periodic full checkpoints include complete model parameters and aggregation buffers, stored in the key value store for recovery. Incremental checkpoints store only parameter differences between versions to reduce storage and network costs. This adaptive approach applies frequent checkpointing when needed while keeping overhead low in stable periods.

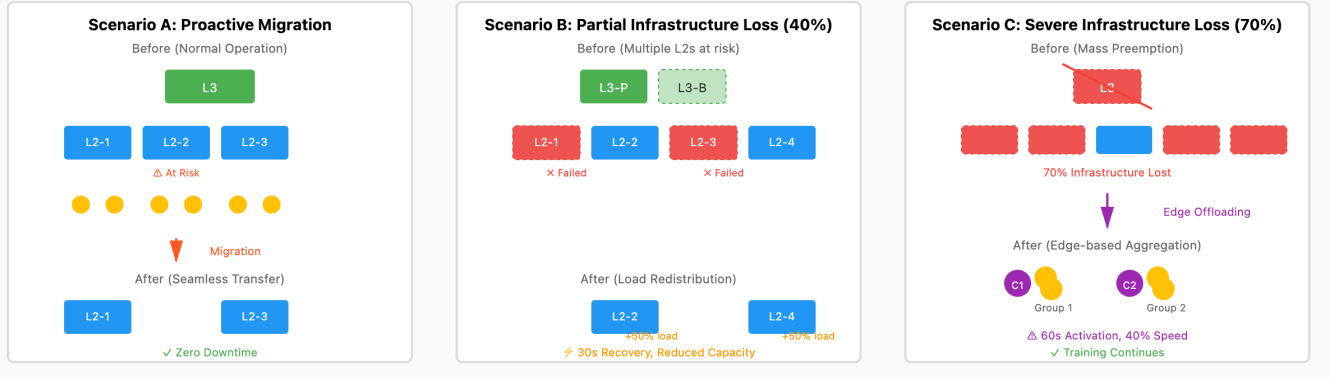


Figure 4: **example fig.** Fault tolerance strategies demonstrating the FL system’s tiered recovery approach.. ‘Scenario A’ shows proactive migration, where an at-risk aggregator transfers state before termination, achieving zero downtime. ‘Scenario B’ illustrates partial infrastructure loss ($\approx 40\%$ aggregators preempted), with surviving aggregators redistributing workload to maintain training at reduced capacity (30s recovery). ‘Scenario C’ depicts severe infrastructure loss ($\approx 70\%$ preempted), triggering edge offloading in which selected clients act as temporary aggregators until cloud resources recover. The system adapts recovery strategy to failure severity, ensuring training continues even under large-scale disruptions.

Migration execution. When preemption risk becomes high or a termination warning is received, the system migrates workloads to stable instances before failure occurs. The resource manager selects target aggregators based on available capacity and proximity to affected clients, then transfers only the most recent state changes to minimize migration time. Depending on resource availability, it may reuse existing instances to avoid startup delay or provision new ones to restore full capacity. These actions are typically completed within the warning period, allowing training to continue with little or no interruption.

3.4.2 Failure Recovery Strategies

(IKK7: this is architectural view of FT mechanism – what happens at each layer...)

While proactive migration (Section 3.4.1) handles most spot preemptions without disruption, multiple aggregator failures can still occur in certain conditions [1, 9, 16, 20]. Such cases include correlated preemptions where many instances in the same availability zone or instance type fail in close succession, large-scale cloud capacity reclaims, and simultaneous high-risk instances that exceed migration capacity. When these events occur, the system adapts its recovery strategy based on failure severity. Figure 4 illustrates three representative scenarios and their corresponding recovery approaches, showing how recovery strategies evolve with increasing failure severity and their impact on recovery time and training throughput.

Partial infrastructure loss. (xxx12: may be change) When several aggregators are preempted but enough capacity remains online, the system combines proactive migration with dynamic load redistribution. The resource manager locates

surviving aggregators with spare capacity and coordinates parallel migrations from affected nodes. To avoid overloading remaining infrastructure, it prioritizes critical workloads such as active client sessions and in-progress aggregations while queuing new client assignments. This controlled degradation allows training to continue at reduced capacity without losing ongoing work.

Edge offloading for severe disruptions. (xxx13: will modify this with sushruth.) When cloud infrastructure loss exceeds the system’s capacity to redistribute workload, e.g., when over 60% (or $\text{xyx} \%$) of aggregators fail simultaneously, the system activates edge offloading. In this mode, selected clients temporarily assume L2 aggregator roles, allowing training to continue without cloud resources. The resource manager identifies candidates based on computational capacity, network bandwidth, and, for mobile devices, battery life. These clients are elected as edge aggregators through a lightweight consensus protocol that accounts for their resources and reliability history. Edge aggregators coordinate to perform partial aggregations and store results in the distributed key-value store when cloud connectivity is available. Training throughput in edge mode is lower, e.g., $\approx 30 - 40\%$ of normal, but it ensures progress instead of halting completely. The system continuously monitors cloud availability and reverts to hierarchical aggregation as soon as sufficient spot instances are provisioned. This rarely invoked capability provides a safeguard for sustaining training under extreme conditions. (xxx14: We should do: 1) update pseudo code, 2) increase novelty with more complexity + leader selection.)

The following pseudocode (Algo. 1) illustrates our fault handling implementation:

Through this tiered approach, e.g., proactive migration for common failures, load redistribution for partial loss, and edge

Algorithm 1 Fault Handling Policy need update...

```

1: function HANDLEPREEMPTION(warning_signal)  ▷
   Proactive migration
2:   doxyz...
3: end function
4: function HANDLEINFRASTRUCTURELOSS(failure_level)
5:   if failure_level < 0.4 then  ▷ Partial loss
6:     do... XXX
7:   else if failure_level < 0.6 then  ▷ Severe loss
8:     do... YYY
9:   else  ▷ Catastrophic loss
10:    do... ZZZ
11:   end if
12: end function
13: function ACTIVATEEDGEOFFLOADING
14:   doxyz...
15: end function

```

offloading for severe disruptions, the system applies the appropriate recovery method for each failure type without adding unnecessary complexity.

3.5 End-to-End System Behavior

(IKK₈: this is new section...)

We now discuss how the system operates as a whole. We illustrate the end-to-end training flow, its adaptive responses under changing conditions, and how it maintains progress under compound failures (Figures 5 and 6).

End-to-end training flow. Figure 5 illustrates the execution of one FL round in our system, which completes in about 185 (or xx) seconds for thousands of clients. The round begins with the resource manager evaluating capacity and provisioning new spot instances if needed. The L3 aggregator then initiates round t and distributes the current global model $v(t)$ to L2 aggregators, which forward it to clients. Clients perform local training and upload updates to their assigned L2s.

Once sufficient updates are collected, L2s execute partial aggregation and synchronize results via the key-value store. The L3 coordinator then performs global aggregation, stores the updated model $v(t+1)$ in the key-value store, and the next round begins.

In parallel, the monitoring component continuously tracks instance health throughout the pipeline (Figure 5, right). If failures are detected, it triggers migration of in-progress workloads to healthy instances, ensuring the round proceeds without interruption. Decision points in the flow adapt dynamically: scaling is triggered when load exceeds thresholds, and aggregation is initiated once enough updates are available.

Adaptive behavior under dynamic conditions. Our system

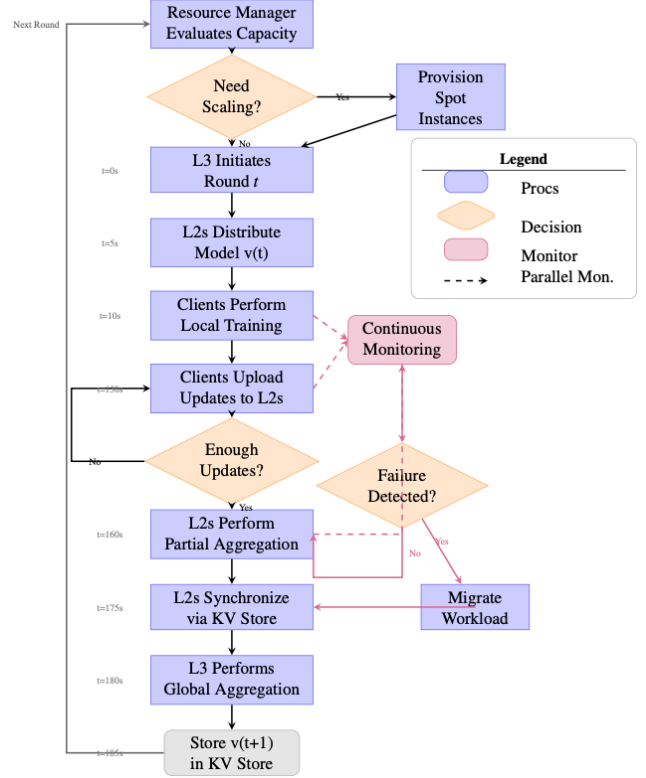


Figure 5: Tikz img. End-to-end training flow for one FL round. The ‘main flow (left)’ shows sequential execution from capacity evaluation to model update storage, taking about 185 (or xyz) seconds for thousands of clients. ‘Parallel monitoring (right)’ continuously checks instance health, detects failures, and triggers migrations without interrupting training. Decision points adapt to dynamic conditions such as scaling needs and aggregation thresholds, before looping to the next round after storing $v(t+1)$

continuously adapts to workload changes without manual intervention. As shown in Figure 6 (top panel), client participation varies significantly over time, from only a few thousand ((xxx₁₅:)) during off-peak hours to more than ten thousand ((xxx₁₆:)) during peak hours. The resource manager tracks load metrics such as client count or CPU utilization per L2 aggregator and provisions new instances when thresholds are exceeded. This ensures smooth scaling—new L2 aggregators are added during client surges and consolidated when demand falls. In parallel, the monitoring component observes instance health and preemption risk, preparing migration targets before failures occur. This dynamic scaling and risk-aware adaptation maintain consistent system performance across fluctuating conditions.

Handling compound failures gracefully. Real-world deployments often face simultaneous disruptions that stress the system beyond individual migrations. Figure 6 (middle and

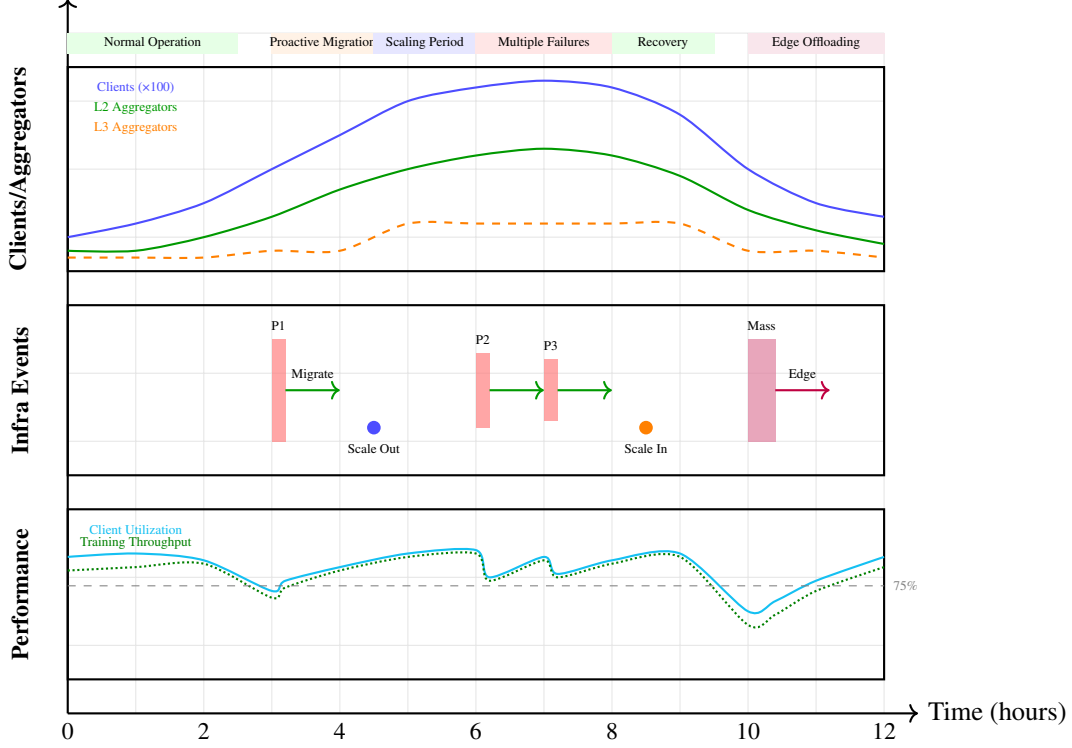


Figure 6: **Integrated system operation during a 12-hour period demonstrating coordinated response to dynamic conditions.** The ‘top panel’ shows client participation varying from 2,000 to 10,000 clients with corresponding L2 aggregator scaling (3-20 instances) and stable L3 count (1-3 instances). The ‘middle panel’ illustrates infrastructure events including individual spot preemptions (P1-P3) successfully handled through proactive migration, and a mass preemption event at hour 10 triggering edge offloading. The ‘bottom panel’ demonstrates system resilience with client utilization remaining above 75% (dashed line) throughout all events, with only temporary degradation during mass failure. Despite experiencing four preemption events and one severe infrastructure loss, the system maintains continuous training through coordinated scaling, migration, and degradation strategies, validating our integrated approach to building reliable FL systems on volatile spot instances.

bottom panels) shows these scenarios, where preemptions occur close together (P1 – P3) or escalate into a severe infrastructure loss around hour 10. In such cases, the resource manager leverages estimated failure probabilities to classify instances by risk and prioritize migration of critical workloads. When multiple failures coincide, the system activates progressive degradation strategies—redistributing load to healthy instances first, then offloading to selected edge devices if cloud resources drop below safe levels. As shown in Figure 6, even during the mass preemption event at hour 10 followed by recovery in hours 11–12, the system maintains client utilization above 75%, ensuring that training never fully stops. This tiered response demonstrates resilience by scaling its defense according to failure severity.

4 Spot Instance Emulator

4.1 Design Overview

Why do we need spot emulator?

1. Large-scale deployment on real spot instances is prohibitively expensive and time-consuming.
2. Need a fast, reproducible (or repeatable), and controllable way to test our system’s perf/behavior on spot instances

So we are making SpotEmul for realistic spot lifetime/behavior modeling, capturing correlated failure, and enabling rapid/repeatable/reproducible experiments.

4.2 Modeling spot dynamics

We explain how emulator models.

- We will explain the dataset first. We are using two different datasets, First one is ISI/USC, which is based on price-based bidding, the second one is using ‘supply-demand’ models like Skypilot and SpotLake. See Table 1.
 1. Price Driven model (pre-2017): Bid-based termination when spot price > user bid
 2. Supply-demand model (post-2017): Capacity-based termination from datacenter dynamics

Table 1: **Public traces used for modeling and validation.** *ISI covers pre-2017 price-based spot behavior; SpotLake summarizes post-2017 supply-demand dynamics with AZ granularity; SkyPilot logs include recent multi-cloud events.*

Source	Period	Spot Model	Scope	Notes
ISI (price hist.)	'14-'15	Price	Global	Legacy; widely referenced
SpotLake	'17-'22	SP	AZ-level	Post 2017 behavior
SkyPilot	'23+	SP	Multi-cloud	Bursty events

Table 2: **Mapping between real-world cloud hierarchy and SpotEmul's emulation model.** *a Kubernetes cluster emulates a data center, each virtual machine represents a physical server, and containers (pods) correspond to individual spot VMs. This abstraction enables us to reproduce large-scale cloud behaviors while running on a single cluster.*

Real World	Emulation World
Data Center	Kubernetes Cluster
Physical Server	Virtual Machine (VM)
Spot VM	Container / Pod

- we will also need to explain how we model lifetime distribution statistically. *i.e.*, Weibull, log-normal, exponential
- What will be preemption (spot revocation) probability over time
- How we model correlated failure across instance types/zones

System-level modeling. See Table 2. We also need to explain time-scaling. *i.e.*, 1 hour (in real-world) to 1 min (in emulation world).

4.3 System Architecture

Architecture and design of the spot emulator, the context is subject to change and need to update the figure.

Overall design. Please see Figure 7.

- Modular framework connecting trace data with live FL jobs on Kubernetes
- Main Controller orchestrates all subcomponents

Trace ingestion and modeling. We will use ISI/USC, Skypilot, and SpotLake logs/traces.

- Inputs: ISI, SpotLake, SkyPilot logs
- Data Extractor derives lifetime distributions & supply-demand models

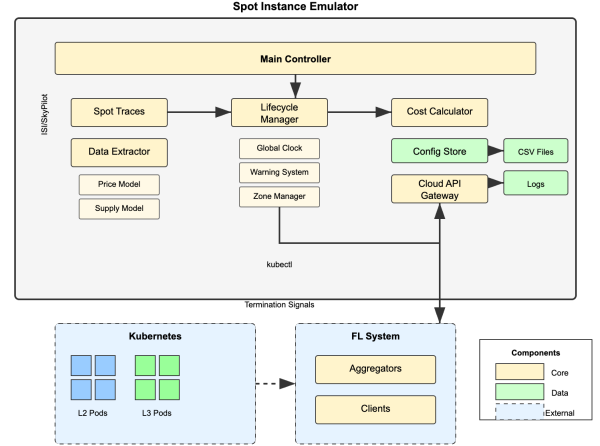


Figure 7: **Architecture of the Spot Instance Emulator.** *SpotEmulator ingests historical spot traces and price/supply models, drives a lifecycle manager to emulate preemptions and warnings, and injects failures into a live FL system deployed on Kubernetes. Its modular design reproduces scenarios such as cascading L2 failures, L3 failover, and mass preemptions triggering edge offloading, enabling evaluation of fault-tolerance strategies at scale without expensive real-world deployments.*

- Also drives failure injections

Lifecycle management.

- We will maintain individual/group of spot lifetime
- Maintains global simulation time
- Issues 2-minute termination warnings
- Coordinates correlated/multi-zone failures

Cost tracking.

- Cost Calculator uses AWS pricing models
- Stores execution costs in config store & logs for later analysis

Failure injection.

- Cloud API Gateway sends termination signals to Kubernetes pods
- Kills L2/L3 aggregators, triggering FL recovery protocols

System integration with FL. (IKK₉; this could be optional)

- L2 and L3 aggregators run as Kubernetes pods
- Emulator injects failures directly into pods
- Exercises migration, redistribution, and edge offloading mechanisms

Key capability. (IKK₁₀; we can write this in a generic way... not specific to scalable FL.)

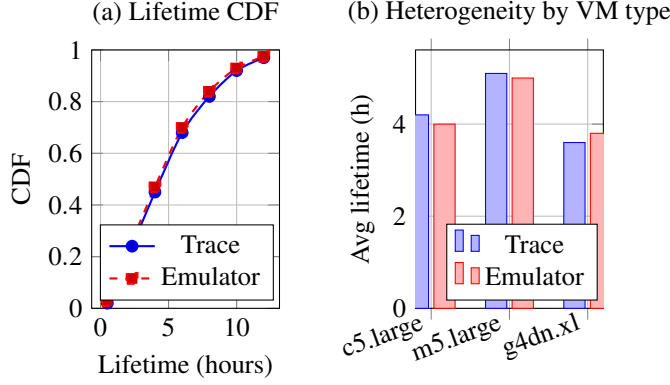


Figure 8: **Validation against public traces.** (a) *Lifetime CDF comparison between emulator output and 180-day AWS production traces. The emulator accurately reproduces the empirical distribution of spot instance lifetimes, with close alignment across short- and long-lived instances.* (b) *Instance-type heterogeneity validation showing that the emulator preserves differences in volatility: GPU instances (e.g., `g4dn.x1`) exhibit much shorter median lifetimes compared to general-purpose instances (e.g., `m5.large`), and these relative trends are faithfully retained in the emulated results. Together, these results confirm that the emulator captures both overall lifetime distributions and instance-type-specific behaviors observed in production.*

- L2 and L3 aggregators run as Kubernetes pods
- Emulator injects failures directly into pods
- Exercises migration, redistribution, and edge offloading mechanisms

4.4 Emulator validation

We will focus on correctness (Figure 8) and speed-up/reproducibility (Figure 9).

5 Evaluation

[evaluation goals] Our goal will be to measure 1) scalability, 2) fault-tolerance, and 3) cost-effectiveness of the proposed FL system under spot dynamics

[Why emulator if needed] Here, we will explain why emulation-based evaluation is necessary (fast, repeatable/reproducible, controlled) if it is necessary

5.1 Evaluation Setup

[cluster setup] Our emulator exec environment will be explained here. For example, we will talk about K8S setup, mapping from emulator world to real-world (refer to Table 2 if needed).

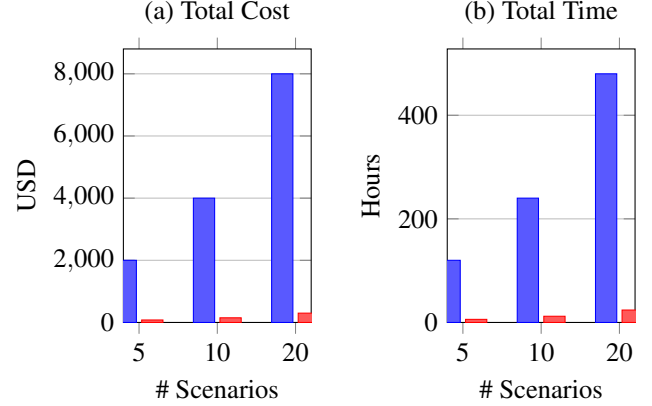


Figure 9: **Cost/time comparison for real deployments vs. spot emulator.** Blue bars: real-spot experiments; red bars: emulator runs. For the same number of scenarios, the emulator completes evaluations faster and at a fraction of the cost. Replace placeholder values with your measurements.

Table 3: **FL models and datasets used in evaluation.** We use diverse vision models from simple CNNs to deep architectures to capture different training characteristics under spot volatility.

Model	Dataset
Single-layer CNN	CIFAR-10
2-layer CNN	CIFAR-10
Logistic Regression	MNIST
ResNet	CIFAR-100
MobileNet	ImageNet
SqueezeNet	MNIST
DenseNet	CIFAR-100

[metrics] We will explain perf metrics here. 1) training throughput (updates/sec, convergence), 2) client utilization, 3) cost efficiency, 4) metrics regarding fault-tolerance, and many more to generate **15+ graphs**.

[FL models and datasets.]

[Aggregation algorithms.] We use six aggregation algos, three lightweight ones like fedavg, fedprox, and fedbuff, and three heavyweight ones like fedadam, fednova, and fedyogi, as explain in Table 4. (IKK₁₁: the math, explanation, and references need to be verified.)

[Workloads] We are going to use two different types of workloads (aka. job arrival patterns), which are 1) step workloads (Figure 10a) and 2) trace-based (Figure 10b) illustrated in Figure 10.

5.2 Evaluation Results

5.2.1 Scalability Evaluation

1) client util, 2) scaling activities (*e.g.*, # of aggs, # of clients),
3) memory utils for six algorithms for six different algorithms

5.2.2 Fault-Tolerance Evaluation

We will add more.

5.2.3 Cost-Efficiency Evaluation

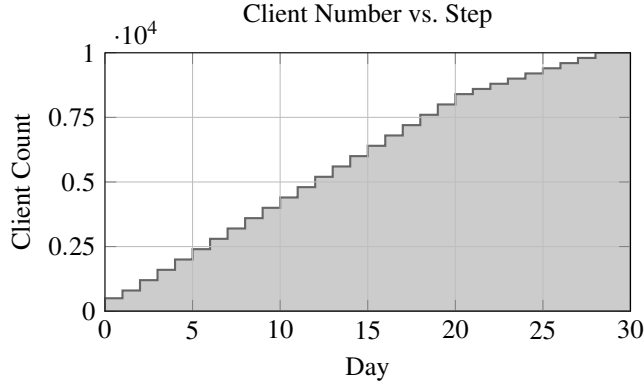
We will add more.

6 Conclusion

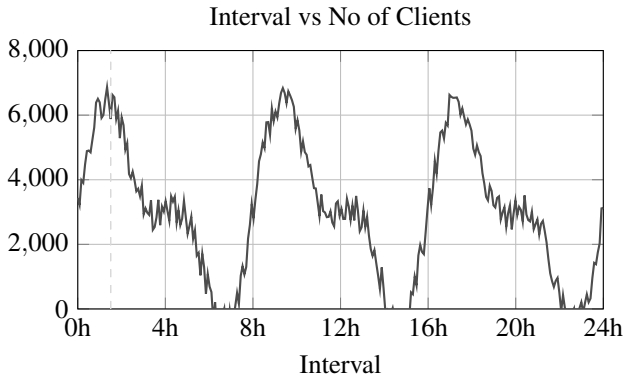
(xxx₁₇: Add Conclusion)

Table 4: **Federated aggregation algorithms evaluated.** We report a brief description and a single key equation that captures each method’s core update. Here n_k is client k ’s data size, $n = \sum_k n_k$, Δ_k is the model delta (or gradient aggregate) returned by client k , τ_k is its local steps, and g_t is the round- t aggregated gradient at the server. **need to check if the math is correct.**

Algorithm	Description	Key feature (equation)
FedAvg [17]	Baseline synchronous averaging of client deltas; weighted by client data size.	$w_{t+1} = w_t + \sum_k \frac{n_k}{n} \Delta_k$
FedProx [14]	Adds a proximal term to stabilize local training under data/step heterogeneity.	Local objective on client k : $\min_w F_k(w) + \frac{\mu}{2} \ w - w_t\ ^2$
FedBuff [18]	Asynchronous/buffered server that aggregates a batch of (possibly stale) updates before applying.	With buffer $\mathcal{B}$ and staleness weight $\alpha(\tau)$: $w \leftarrow w + \frac{1}{\sum_{i \in \mathcal{B}} \alpha(\tau_i)} \sum_{i \in \mathcal{B}} \alpha(\tau_i) \Delta_i$
FedNova [22]	Normalizes client updates by local optimization length to correct objective mismatch.	$\Delta^{\text{nova}} = \sum_k \frac{n_k}{n} \frac{\Delta_k}{\tau_k}$, $w_{t+1} = w_t + \Delta^{\text{nova}}$
FedAdam [19]	Applies Adam at the server to the aggregated gradient for faster convergence.	$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$, $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$, $w_{t+1} = w_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon}$
FedYogi [19]	Uses Yogi (Adam variant) at the server to control variance via conservative second-moment updates.	$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$, $v_t = v_{t-1} - (1 - \beta_2) \text{sign}(v_{t-1} - g_t^2) \odot g_t^2$, $w_{t+1} = w_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon}$



(a) Step Load (tikz)



(b) Trace-Driven. (tikz - need data)

Figure 10: **Workload traces used for evaluation demonstrating two distinct client participation patterns.** (a)/(c) *Step Load*: Synthetic workload with monotonically increasing client count from 500 to 10,000 over 30 days, used to evaluate system scalability under predictable growth. Each step represents approximately 300-400 additional clients joining the FL training. (b)/(d) *Trace-Driven*: Real-world diurnal pattern based on mobile device availability traces showing typical daily fluctuations between 1,000 clients (off-peak) and 7,000 clients (peak), with 96 intervals per day (15-minute granularity). The pattern exhibits characteristic peaks during business hours and valleys during night hours, representative of actual FL deployment scenarios.

References

- [1] Spot Instance interruptions. <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-interruptions.html>, 2025. [ONLINE].
- [2] Daniel J. Beutel, Taner Topal, Akhil Mathur, Xinchu Qiu, Titouan Parcollet, and Nicholas D. Lane. Flower: A Friendly Federated Learning Research Framework. *CoRR*, abs/2007.14390, 2020.
- [3] Kallista A. Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingerman, Vladimir Ivanov, Chloé Kiddon, Jakub Konečný, Stefano Mazzocchi, Brendan McMahan, Timon Van Overveldt, David Petrou, Daniel Ramage, and Jason Roselander. Towards Federated Learning at Scale: System Design. In *Proceedings of the Second Conference on Machine Learning and Systems, SysML 2019, Stanford, CA, USA, March 31 - April 2, 2019*.
- [4] Cheng-Wei Ching, Xin Chen, Taehwan Kim, Bo Ji, Qingyang Wang, Dilma Da Silva, and Liting Hu. Totoro: A Scalable Federated Learning Engine for the Edge. In *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024*.
- [5] Katharine Daly, Hubert Eichner, Peter Kairouz, H. Brendan McMahan, Daniel Ramage, and Zheng Xu. Federated Learning in Practice: Reflections and Projections. *CoRR*, abs/2410.08892, 2024.
- [6] Supriyo Ghosh, Manish Shetty, Chetan Bansal, and Suman Nath. How to fight production incidents?: an empirical study on a large-scale cloud service. In *Proceedings of the 13th Symposium on Cloud Computing, SoCC 2022, San Francisco, California, November 7-11, 2022, 2022*.
- [7] Phillipa Gill, Navendu Jain, and Nachiappan Nagappan. Understanding network failures in data centers: measurement, analysis, and implications. In *Proceedings of the ACM SIGCOMM 2011 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications, Toronto, ON, Canada, August 15-19, 2011, 2011*.
- [8] Haryadi S. Gunawi, Mingzhe Hao, Riza O. Suminto, Agung Laksono, Anang D. Satria, Jeffry Adityatama, and Kurnia J. Eliazar. Why Does the Cloud Stop Computing? Lessons from Hundreds of Service Outages. In *Proceedings of the Seventh ACM Symposium on Cloud Computing, Santa Clara, CA, USA, October 5-7, 2016, 2016*.
- [9] Aaron Harlap, Andrew Chung, Alexey Tumanov, Gregory R. Ganger, and Phillip B. Gibbons. Tributary: spot-dancing for elastic services with latency SLOs. In *Proceedings of the 2018 USENIX Annual Technical Conference, USENIX ATC 2018, Boston, MA, USA, July 11-13, 2018*.
- [10] Chaoyang He, Songze Li, Jinhyun So, Mi Zhang, Hongyi Wang, Xiaoyang Wang, Praneeth Vepakomma, Abhishek Singh, Hang Qiu, Li Shen, Peilin Zhao, Yan Kang, Yang Liu, Ramesh Raskar, Qiang Yang, Murali Annavaram, and Salman Avestimehr. FedML: A Research Library and Benchmark for Federated Machine Learning. *CoRR*, abs/2007.13518, 2020.
- [11] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista A. Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, Rafael G. L. D'Oliveira, Hubert Eichner, Salim El Rouayheb, David Evans, Josh Gardner, Zachary Garrett, Adrià Gascón, Badri Ghazi, Phillip B. Gibbons, Marco Gruteser, Zaid Harchaoui, Chaoyang He, Lie He, Zhouyuan Huo, Ben Hutchinson, Justin Hsu, Martin Jaggi, Tara Javidi, Gauri Joshi, Mikhail Khodak, Jakub Konečný, Aleksandra Korolova, Farinaz Koushanfar, Sanmi Koyejo, Tancrède Lepoint, Yang Liu, Prateek Mittal, Mehryar Mohri, Richard Nock, Ayfer Özgür, Rasmus Pagh, Hang Qi, Daniel Ramage, Ramesh Raskar, Mariana Raykova, Dawn Song, Weikang Song, Sebastian U. Stich, Ziteng Sun, Ananda Theertha Suresh, Florian Tramèr, Praneeth Vepakomma, Jianyu Wang, Li Xiong, Zheng Xu, Qiang Yang, Felix X. Yu, Han Yu, and Sen Zhao. Advances and Open Problems in Federated Learning. *Foundations and Trends in Machine Learning*, 14(1-2):1–210, 2021.
- [12] Fan Lai, Yinwei Dai, Sanjay Sri Vallabh Singapuram, Jiachen Liu, Xiangfeng Zhu, Harsha V. Madhyastha, and Mosharaf Chowdhury. FedScale: Benchmarking Model and System Performance of Federated Learning at Scale. In *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA, 2022*.
- [13] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. Federated Learning: Challenges, Methods, and Future Directions. *IEEE Signal Processing Magazine*, 37(3):50–60, 2020.
- [14] Tian Li, Anit Kumar Sahu, Manzil Zaheer, Maziar Sanjabi, Ameet Talwalkar, and Virginia Smith. Federated Optimization in Heterogeneous Networks. In *Proceedings of the Third Conference on Machine Learning and Systems, MLSys 2020, Austin, TX, USA, March 2-4, 2020*.

- [15] Lumin Liu, Jun Zhang, Shenghui Song, and Khaled B. Letaief. Client-Edge-Cloud Hierarchical Federated Learning. In *2020 IEEE International Conference on Communications, ICC 2020, Dublin, Ireland, June 7-11, 2020*, 2020.
- [16] Ziming Mao, Tian Xia, Zhanghao Wu, Wei-Lin Chiang, Tyler Griggs, Romil Bhardwaj, Zongheng Yang, Scott Shenker, and Ion Stoica. SkyServe: Serving AI Models across Regions and Clouds with Spot Instances. In *Proceedings of the Twentieth European Conference on Computer Systems, EuroSys 2025, Rotterdam, The Netherlands, 30 March 2025 - 3 April 2025*.
- [17] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, AISTATS 2017, 20-22 April 2017, Fort Lauderdale, FL, USA, 2017*.
- [18] John Nguyen, Kshitiz Malik, Hongyuan Zhan, Ashkan Yousefpour, Mike Rabbat, Mani Malek, and Dzmitry Huba. Federated Learning with Buffered Asynchronous Aggregation. In *International Conference on Artificial Intelligence and Statistics, AISTATS 2022, 28-30 March 2022, Virtual Event*.
- [19] Sashank J. Reddi, Zachary Charles, Manzil Zaheer, Zachary Garrett, Keith Rush, Jakub Konečný, Sanjiv Kumar, and Hugh Brendan McMahan. Adaptive Federated Optimization. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*.
- [20] Prateek Sharma, Stephen Lee, Tian Guo, David E. Irwin, and Prashant J. Shenoy. SpotCheck: designing a derivative IaaS cloud on the spot market. In *Proceedings of the Tenth European Conference on Computer Systems, EuroSys 2015, Bordeaux, France, April 21-24, 2015*.
- [21] Guosai Wang, Lifei Zhang, and Wei Xu. What Can We Learn from Four Years of Data Center Hardware Failures? In *47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2017, Denver, CO, USA, June 26-29, 2017*, 2017.
- [22] Jianyu Wang, Qinghua Liu, Hao Liang, Gauri Joshi, and H. Vincent Poor. Tackling the Objective Inconsistency Problem in Heterogeneous Federated Optimization. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*.
- [23] Zheng Xu, Yanxiang Zhang, Galen Andrew, Christopher A. Choquette-Choo, Peter Kairouz, H. Brendan McMahan, Jesse Rosenstock, and Yuanbo Zhang. Federated Learning of Gboard Language Models with Differential Privacy. In *Proceedings of the The 61st Annual Meeting of the Association for Computational Linguistics: Industry Track, ACL 2023, Toronto, Canada, July 9-14, 2023*, 2023.
- [24] Chengxu Yang, Qipeng Wang, Mengwei Xu, Zhenpeng Chen, Kaigui Bian, Yunxin Liu, and Xuanzhe Liu. Characterizing Impacts of Heterogeneity in Federated Learning upon Large-Scale Smartphone Data. In *WWW '21: The Web Conference 2021, Virtual Event / Ljubljana, Slovenia, April 19-23, 2021*.

7 Todo

1. Listup contributions and match up with current writing
2. discussion with Susurhth.

SpotLight: Title is TBD

Anonymous Author(s)

Submission Id: #xxx₁

Abstract

CCS Concepts

• Computer systems organization → Cloud computing; • Software and its engineering → Scheduling; Software performance; • Computing methodologies → Planning and scheduling.

Keywords

xxx, yyy, zzz

1 Introduction

Hybrid spot/on-demand strategies [11, 32] have been extensively studied.

2 Motivation

Federated learning (FL) is now widely used for training machine learning models without centralizing training data [19]. As client populations grow, the infrastructure responsible for aggregating model updates must scale accordingly. Production FL deployments already coordinate tens of millions of devices, with system designers targeting deployments at the scale of billions of devices [3], placing significant demand on the aggregation layer. Existing frameworks such as Flower [2], IBM FL [18], FATE [17], and Papaya [8] are designed as single aggregator systems. In production FL deployments, the aggregation layer is responsible for managing the lifecycle of connecting clients, selecting and forwarding devices into training rounds, and combining their model updates into a global model. This breadth of responsibility places significant resource demands on a single aggregator, and as the number of clients grows, single aggregators become bottlenecks that increase the wall clock time of training, as clients are forced to wait longer for the aggregator to complete a round before receiving the next set of updates [10].

Papaya [8] and Flower [2] are both single-aggregator FL systems, though their execution models differ fundamentally. FL payload sizes vary widely, from 22MB for a model with 11 million parameters [5] to over 140GB for large language models [21], and in a single-aggregator system every update must be received, validated, and processed by that one node. As client load grows, this concentration of responsibility makes the aggregator a bottleneck. Flower’s synchronous design exposes this limitation at modest scale, causing training to stall Figure 1. Papaya’s asynchronous execution model raises this threshold considerably, but a similar failure mode emerges at higher client load Figure 2.

A straightforward response to the limitations of a single aggregator is to add aggregators as client load increases. The stateful nature of FL aggregation, however, makes this non-trivial. Horizontally scaling the aggregation layer introduces a set of problems absent from the single-aggregator case.

The first such problem is maintaining global model consistency across aggregator instances. Each aggregator receives updates from only a subset of clients, producing a sub-global model accumulated

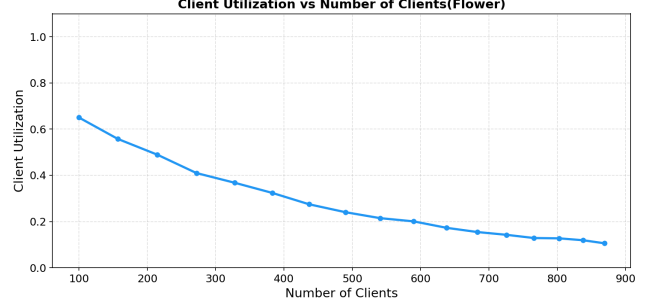


Figure 1: Client utilization for Flower as the number of participating clients scales from 100 to 895, configured with a 95% client quorum and 10MB payloads. Client Utilization declines monotonically from 0.65 to below 0.12, at which point Flower fails entirely as the aggregator can no longer keep pace with incoming client updates.

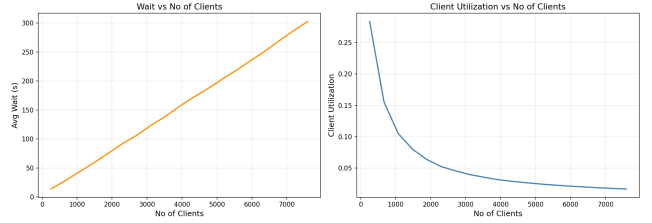


Figure 2: Client wait time and client utilization for Papaya under a varying client load. As client load increases, average wait time grows linearly from 15s to over 300s, while client utilization collapses from 0.28 to near zero, indicating that clients spend the vast majority of their time waiting rather than training.

from the subset of clients. The sub-global models produced by individual aggregators would require an additional step to produce a consistent global model that incorporates updates from all clients, a coordination requirement inherent to any system that partitions shared state across multiple nodes [15].

The second challenge with aggregator scaling is model staleness. Due to the differences in the subset of updates that aggregators individually receive, the model version a client trains on may diverge from the current global model. This divergence causes a subset of clients to take longer to converge as staleness increases [31]. The third challenge with aggregator scaling is scheduling client updates across aggregators, which is difficult for two reasons. First, aggregation is computationally expensive, so a scheduler that ignores load can bottleneck a single aggregator even while others sit idle. Second, incoming client updates must be placed on an aggregator holding the current global model rather than an older one, to

Design Insight 1: Scaling FL aggregation requires more than replicating a single aggregator; consistency, staleness, scheduling, and state propagation costs multiply with each instance added. An explicit step is further required to merge sub-global models into a consistent global model. This motivates a two-tiered architecture in which coordination is bounded within each tier and a dedicated aggregation step merges sub-global models into a single global model.

avoid wasting the client’s training effort on a model the system has already moved past. The fourth challenge with aggregator scaling is propagating a common state across aggregators following aggregation. The updated global model, along with server-side optimizer state such as momentum or adaptive learning-rate terms maintained alongside the model, must be propagated to every aggregator [23]. The synchronization of the global model and server-side optimizer state must occur throughout the FL process, making it a persistent challenge. These four challenges illustrate that naively scaling single-aggregator systems horizontally in response to client load is insufficient for maintaining model consistency and client performance. The FL system architecture must therefore be redesigned from a single-aggregator model to one with FL-specific coordination, synchronization, and scheduling.

Aggregators in FL systems are deployed on always-on, on-demand cloud instances [10, 22]. To quantify the cost, we measure a median per-round aggregator time of 732 seconds on a p3.2xlarge instance and extrapolate to the 1,198 rounds required for FedAvg convergence on ResNet-18 trained on CIFAR-100 [27], yielding a projected training duration of approximately 244.5 hours. Figure 3 shows the resulting aggregator cost under three scenarios: on-demand pricing, ideal spot pricing assuming no preemptions, and spot pricing without fault tolerance where each preemption forces a full job restart. Spot instances offer a substantially cheaper alternative to on-demand instances [28, 30], being up to 88% cheaper [14]; however, spot instances can be preempted at any time by the cloud provider and come with no availability guarantees [9, 26]. While ideal spot pricing reduces aggregator cost by 88.7% over on-demand, preemptions without fault tolerance force full job restarts, inflating the realized spot cost to only 13.5% cheaper than on-demand, nearly erasing the spot pricing advantage altogether. Therefore, realizing the full cost benefits of spot instances requires fault tolerance mechanisms that preserve training state across preemptions [4, 24]. While techniques such as checkpointing and instance migration are commonly adopted for this purpose, each presents fundamental limitations when applied to FL aggregation systems. We identify three such limitations below. The first limitation is to do with Checkpointing, it is a commonly adopted FT technique that persists application state to durable storage, enabling recovery following a failure. In the context of FL aggregation, a naive checkpointing strategy saves the global model state after every aggregation update. However, as the number of aggregation updates grows, the cumulative I/O overhead of checkpointing every update grows proportionally. At scale, this overhead becomes a significant bottleneck, stalling the aggregator and degrading training throughput.

The second limitation is to do with migration, it is a FT technique that transfers the state of a failed instance to a newly provisioned

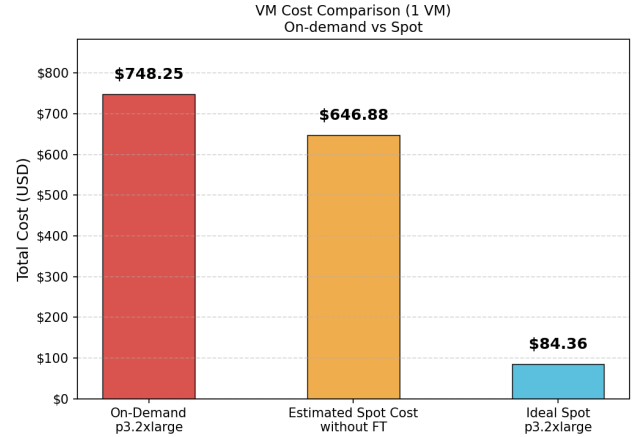


Figure 3: Total aggregator cost to train ResNet-18 on CIFAR-100 to convergence (1,198 rounds [27]) on a single p3.2xlarge instance. On-demand pricing at \$3.06/hr costs \$748.25 for 244.5 hours of uninterrupted training. Spot preemptions, in the absence of fault tolerance, force full job restarts and inflate training cost to \$646.88 at \$0.345/hr, a 7.6x overhead over the ideal spot cost of \$84.36 assuming uninterrupted execution.

Design Insight 2: Checkpointing, instance migration, and provider termination signals are each insufficient for FL aggregation on spot infrastructure. Effective fault tolerance requires decoupling aggregator state from instance lifecycle and acting on failure likelihood independently of provider-issued signals.

replacement, allowing the system to resume operation. In the context of FL aggregation, this involves transferring the in-progress sub-global model, optimizer state, and client update buffer to a replacement aggregator following a spot preemption. However, cloud VM provisioning is not instantaneous; prior work has demonstrated that newly provisioned VMs incur substantial startup latency [7, 29]. During this provisioning window, the aggregator is unavailable, and for a system processing a large volume of client updates, this downtime results in lost updates, stalled training and increased time to reach target accuracy.

Finally Spot VM lifetimes are non-deterministic and have no prior knowledge of when preemption will occur. Although cloud providers issue a termination signal prior to reclamation, the notice period varies considerably across providers; AWS issues a two-minute warning [1] while GCP issues a 30-second warning [6]. The state transfer overhead inherent to FL aggregation systems may exceed the available notice window, rendering provider termination signals an insufficient basis for guaranteeing successful migration prior to instance reclamation.

3 SpotLight System Overview

SpotLight enables federated learning to scale elastically on spot infrastructure without sacrificing aggregator performance or cost efficiency. This section overviews the architecture of SpotLight

Design Insight 3: Scaling the aggregator is necessary to sustain throughput under growing client load, yet multiple on-demand instances compound infrastructure cost. Spot instances present a viable path to containing this cost, but as existing fault tolerance mechanisms are insufficient to guard against preemptions, realized spot costs inflate to near on-demand levels, erasing the pricing advantage entirely. A framework that natively co-designs horizontal scaling with spot-aware fault tolerance is therefore required to achieve both performance and cost efficiency.

(§3.1) and the three design principles behind it (§3.2): *two-tiered aggregation* that scopes each aggregator to its assigned clients and combines partial models into a consistent global model, *proactive fault tolerance* that migrates workloads and checkpoints aggregator state upon detecting failure likelihood, and *elastic spot provisioning* that treats scalability and fault tolerance as co-equal concerns to fully exploit spot infrastructure.

3.1 SpotLight Architecture

Fig. 4 shows the architecture of SpotLight. The system comprises three main components: the **Resource Manager (RM)**, the **aggregation layer**, and the **Performance Manager**. The aggregation layer consists of **Sub-Global Aggregators**, **Global Aggregators**, and **Failover Edge Aggregators**, which extend the aggregation hierarchy to the edge when cloud aggregators are unavailable. All external client connections are fronted by Envoy, and MongoDB persists coordination state shared across all components.

The RM is itself horizontally scalable, deployed as multiple Kubernetes instances, and internally comprises four subcomponents. The **Scheduler** manages client membership, assigns clients to Sub-Global Aggregator cohorts, and schedules incoming updates. The **Scaler** provisions and deprovisions Sub-Global and Global Aggregators in response to signals from the Performance Manager, forming a closed loop that drives elastic scaling. The **Migrator** monitors aggregator failure likelihood and, upon detecting an at-risk instance, determines an appropriate migration target, either a healthy cloud aggregator or a Failover Edge Aggregator, and triggers workload migration and state checkpointing on the at-risk instance. The **Leader Elector** runs Raft consensus across RM instances to elect a single leader with sole authority over scaling and migration decisions, allowing the RM to scale horizontally without becoming a single point of failure.

SpotLight Execution Flow. Clients submit updates through Envoy to the RM, where the Scheduler assigns each update to a Sub-Global Aggregator cohort. Sub-Global Aggregators accumulate updates from their assigned clients and produce sub-global models, which Global Aggregators merge into a single consistent global model. Concurrently, the Performance Manager observes load across both aggregation tiers and signals the Scaler to provision or deprovision aggregators as demand shifts. When the Migrator detects elevated failure likelihood on an aggregator, the elected leader directs workload migration to a healthy target and checkpoints the at-risk instance’s current state. If no healthy cloud aggregators are available, a Failover Edge Aggregator assumes the role. The RM redelivers

any in-flight updates to the replacement, and training continues without restarting.

3.2 Three Design Principles

We further elaborate on the three design principles of SpotLight, explaining how each addresses the insights identified in §2.

Two-tiered aggregation. Scaling aggregators increases coordination and synchronization overhead. SpotLight manages this through a two-tiered aggregation structure in which each Sub-Global Aggregator handles only its assigned subset of clients, and Global Aggregators receive only a fixed number of sub-global models regardless of total client count. This bounds the coordination scope of each tier independently, allowing both tiers to scale without overhead growing proportionally with the number of clients or aggregators. Global Aggregators then merge the fixed set of sub-global models into a single consistent global model, providing an explicit aggregation step that scales independently of client count.

Proactive fault tolerance. Naive fault tolerance mechanisms that rely on provider termination signals, periodic checkpointing, or restarting lost work are insufficient for FL aggregation on spot infrastructure. SpotLight instead uses a Kaplan-Meier survival model [12] derived from publicly available spot VM lifetime traces [30] to estimate failure likelihood ahead of preemption, triggering workload migration and state checkpointing before an instance is lost. This ensures that aggregation continues without losing any in-flight updates.

Elastic spot provisioning. On-demand instances make sustained horizontal scaling economically prohibitive. SpotLight provisions its aggregation layer on spot instances, but the cost advantage of spot can only be realized if the system can absorb aggregator disruptions without stalling throughput. Scalability and fault tolerance are therefore co-equal concerns in SpotLight, working together simultaneously to sustain aggregator throughput and preserve the cost efficiency of spot infrastructure at scale.

4 System Components

Building on the three design principles established in §3.2, this section describes how SpotLight realizes them in practice. The aggregation tiers realize two-tiered aggregation through independently scalable Sub-Global and Global Aggregators. The Resource Manager handles scheduling, orchestrates scaling across both tiers, and migrates workloads before preemption occurs, preserving training continuity and the cost efficiency of spot infrastructure. We begin with the update lifecycle (§4.1), which establishes the end-to-end path a client update follows through the system, then describe each component in detail.

4.1 Update Lifecycle

In SpotLight, client updates flow through two aggregation tiers, accumulating into sub-global models before being merged into the global model. Clients maintain persistent gRPC connections to Envoy, which routes them to the Resource Manager. The RM forwards client updates to the appropriate Sub-Global Aggregator asynchronously in the background, decoupling client-facing latency from aggregator throughput. The Sub-Global Aggregator

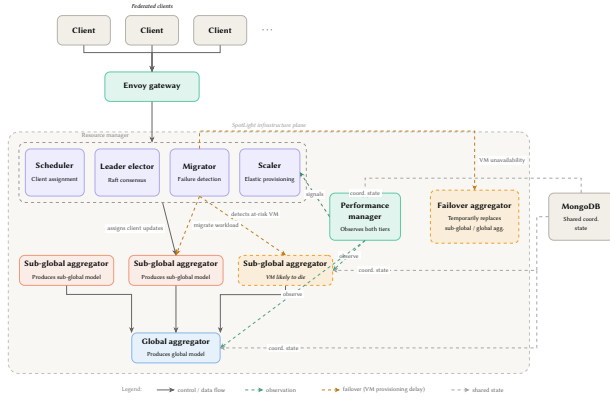


Figure 4: SpotLight system architecture. The Resource Manager coordinates a two-tier aggregation hierarchy backed by MongoDB for shared coordination state. The Performance Manager observes both aggregation tiers and signals the Scaler for elastic provisioning. The Migrator proactively detects aggregators on at-risk VMs and redirects their workload to a standby failover aggregator to bridge VM provisioning delay.

accumulates updates in an in-memory buffer and triggers aggregation once the floor threshold is met. The resulting sub-global model is pushed to the Global Aggregator, which incorporates it into the global model. Sub-Global Aggregators pull the updated global model and propagate it down the hierarchy to clients.

4.2 Two-Tiered Aggregation

SpotLight distributes aggregation across two tiers, allowing each tier to scale independently as client participation grows.

Aggregators operate at two distinct points in the hierarchy with different responsibilities. Sub-Global Aggregators receive updates from the RM, aggregate them into a sub-global model, and ensure clients receive the latest available model. Global Aggregators receive sub-global models from all Sub-Global Aggregators and merge them into a single global model. Both tiers adopt FedBuff’s [20] buffered aggregation model, but use the buffer as a floor rather than a strict barrier, triggering aggregation as soon as the minimum threshold is met, avoiding stalls from incomplete batches and improving aggregator throughput. Sub-Global Aggregator workload scales directly with client participation and can vary significantly as clients join or leave training, while Global Aggregator workload is bounded by the number of Sub-Global Aggregators, making it more stable. This difference in workload characteristics motivates distinct scaling policies at each tier.

5 Implementation

5.1 SpotLight Implementation

SpotLight is implemented in Python across 19.2k lines of code, reflecting the breadth of its control plane, fault tolerance, and scheduling components, and is deployed on a Kubernetes cluster. SpotLight utilizes third-party libraries including Motor, an asynchronous MongoDB driver, gRPC for inter-component communication, NumPy

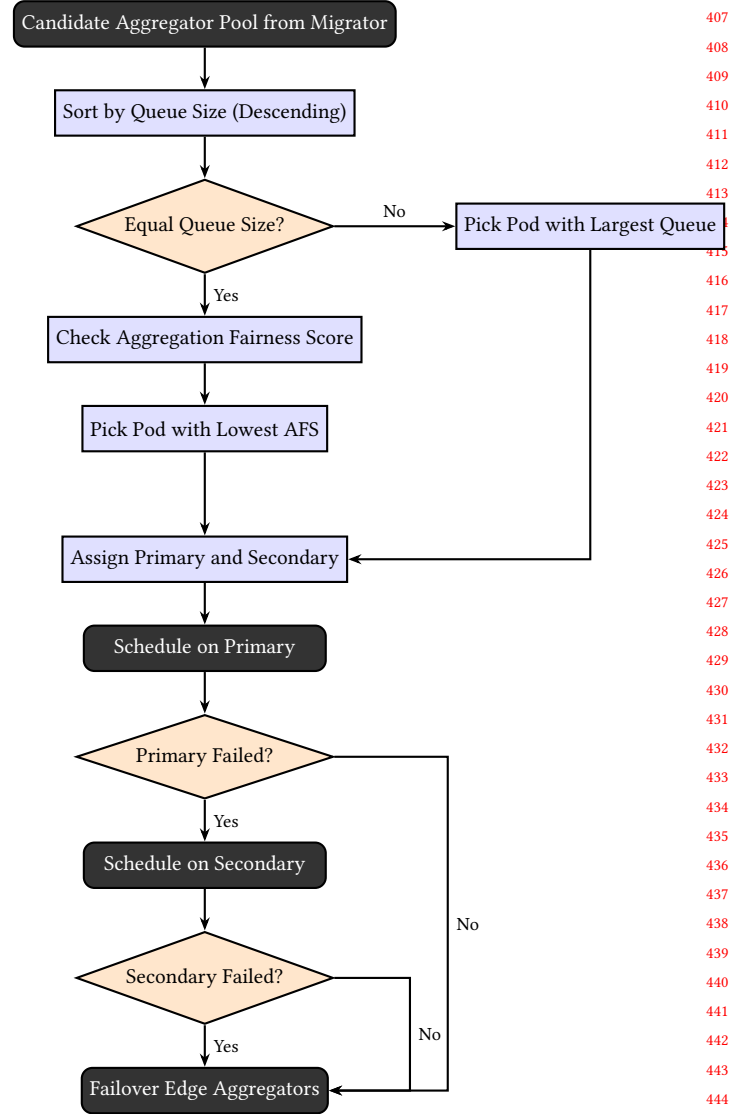


Figure 5: Scheduling algorithm. SpotLight prioritizes aggregators closest to their aggregation threshold, using the Aggregation Fairness Score (AFS) to prevent load concentration, with a two-tier failover through secondary aggregators and Failover Edge Aggregators.

for algorithm implementations, and the Kubernetes Python client for cluster management. SpotLight uses MongoDB for coordination, state management, and client lifecycle tracking. System components are deployed as StatefulSets, which provide stable network identities necessary for gRPC-based inter-component communication, and communicate exclusively over gRPC, chosen over REST for its HTTP/2-native multiplexing and bidirectional streaming, which eliminates per-request connection overhead on the critical path of persistent client-to-RM streams. All inbound client connections are routed through an Envoy gateway, selected for its native gRPC

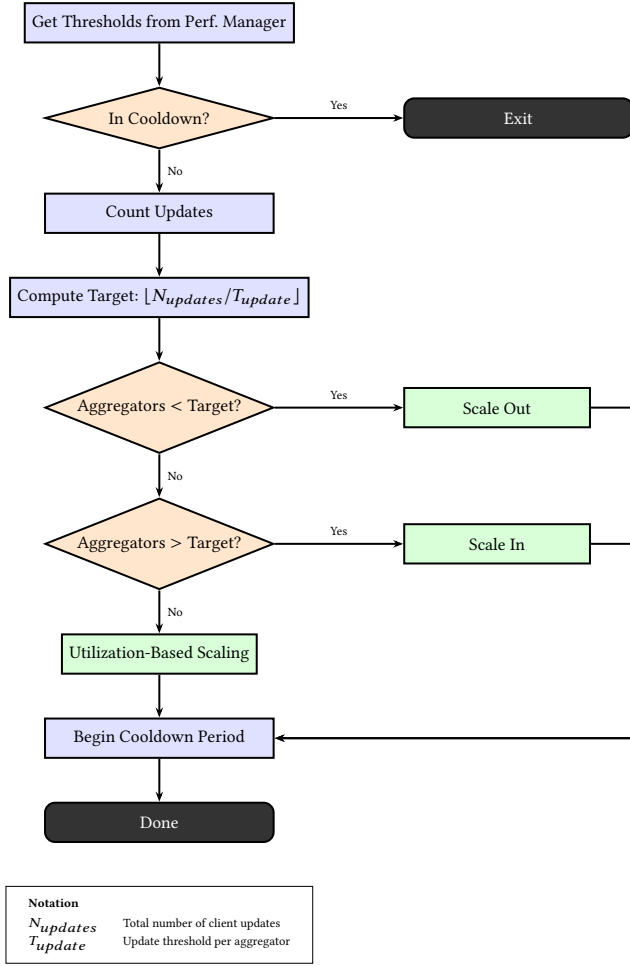


Figure 6: Aggregator Scaling Algorithm. The autoscaler derives a target aggregator count from incoming update load and adjusts capacity accordingly, falling back to utilization-based scaling when the count is already on target. A cooldown period gates all scaling actions to prevent oscillation.

support and connection load balancing. SpotLight was hosted on Chameleon Cloud [13] during evaluation

5.2 Model State Management

SpotLight uses MongoDB as its shared persistence layer, storing coordination state accessible across all RM replicas and aggregator instances. Under normal operation, client updates and global model state are managed in-memory across the Sub-Global and Global aggregation tiers. Upon elevated failure likelihood, signaled by the Kaplan-Meier failure estimator (Section ??), the aggregator persists its in-memory buffer of client updates, current model weights, and server-side optimizer state to MongoDB. For stateful algorithms such as FedAdam [23], server-side optimizer state is additionally persisted; stateless algorithms such as FedAvg [19] require only the model weights. While the aggregator persists its accumulated state, the RM independently tracks the delivery status of all in-flight

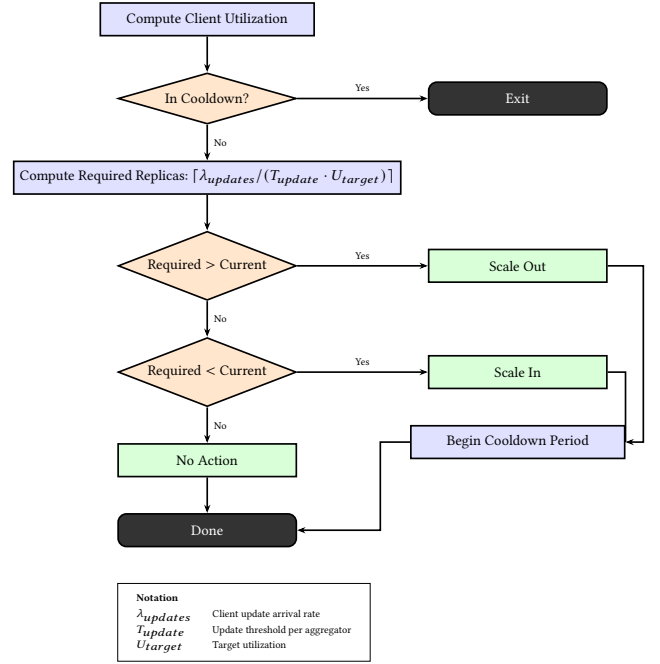


Figure 7: Utilization-Based Scaling Algorithm. A fallback path that adjusts aggregator count when client utilization signals overload, even when update-based thresholds have determined no scaling is required.

client updates. As described in Section 4, the RM redelivers any unacknowledged updates to the next available aggregator selected by the Scheduler. All aggregators, whether newly provisioned or already running, maintain a synchronized view of training state by continuously pulling the latest model parameters and state from MongoDB. This design ensures no client updates are lost due to preemptions or transient network failures.

5.3 Aggregator Failover

Spot VM provisioning introduces non-trivial startup latency that can exceed the preemption warning window provided by cloud providers [7, 29], during which training would otherwise stall. SpotLight bridges this window using a statically configured pool of Failover Edge Aggregators that assume the role of the unavailable cloud aggregator for the duration of the provisioning delay. The RM polls aggregator health at 0.1 second intervals, minimizing the latency between detecting unavailability and activating a Failover Edge Aggregator. The RM determines which aggregator tier has become unavailable and assigns the corresponding Sub-Global or Global Aggregator role to the Failover Edge Aggregator. Failover Edge Aggregators pull the latest model parameters and state from MongoDB before accepting client updates, consistent with the persistence model described in Section 5.2. Once the newly provisioned cloud aggregator is detected as ready, the RM redirects traffic back and deactivates the edge instance.

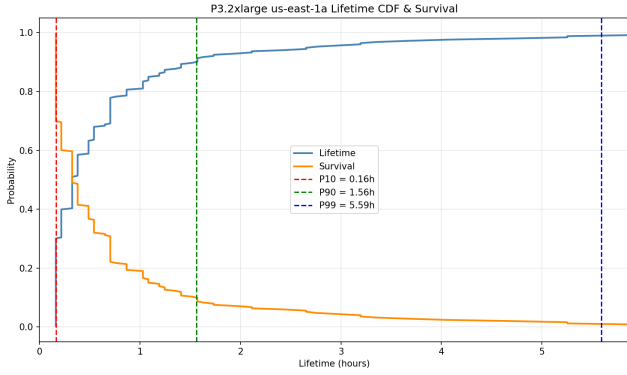


Figure 8: Lifetime CDF and survival function $S(t)$ for p3.2xlarge instances in us-east-1a, derived from availability traces in [30]. The distribution is heavy-tailed: 10% of instances are preempted within 0.16 hours (P10), 90% within 1.56 hours (P90), and 99% within 5.59 hours (P99). The Spot Emulator samples instance lifetimes from this CDF to replay preemption events during evaluation.

5.4 Spot Emulator

A key feature of spot instances is their variability in preemption behavior, driven by factors including region, availability zone, instance type, temporal market conditions and cloud providers [4, 16, 25]. This variability across configurations makes real-world evaluation of application behavior on spot instances difficult to reproduce consistently and cost-prohibitive at scale, as comprehensive coverage across regions and instance types requires extensive testing for long periods of time. Therefore, to evaluate SpotLight under realistic spot conditions, we developed the Spot Emulator, a high-fidelity trace-driven emulator that replays spot VM behavior using publicly available spot availability traces [14, 30]. The emulator is designed around two key goals. First, high fidelity: the emulator replays preemption events derived from real-world availability traces, ensuring emulated behavior reflects realistic spot market conditions. Second, controller-agnostic design: the modular architecture of the Spot Emulator allows it to support Spot VM behavior replay with different cloud services providers and deploy on non-Kubernetes platforms with minimal changes to the emulator.

The lifetime distribution used to replay spot behavior is derived from the availability traces in [30]. The availability trace is used to derive a CDF, which allows the emulator to reliably replay spot instance lifetime dynamics. To ensure that evaluation cost is realistic, price traces obtained from [14] are used to generate realistic execution costs. As shown in Figure 8, survival probability decreases monotonically with instance lifetime, reflecting the inherent unreliability of spot instances over time.

The Spot Emulator consists of five components. The first is the *Distribution Extractor*, which filters the availability trace by region, availability zone, and OS, and derives the lifetime CDF from the filtered data. The second is the *Lifecycle Manager*, which samples a lifetime for each newly provisioned VM from the CDF within the P10–P90 range, excluding tail outliers that represent atypically

short or long-lived instances, and maintains an internal clock tracking the lifetime of all active spot VMs. Once a VM’s assigned lifetime expires, the Lifecycle Manager triggers the third component, the *Executor*, which terminates the instance via the Kubernetes API. The fourth component is the *Control Plane*, which manages the execution order of the other components and exposes configuration parameters to the user. The fifth component is the *Cost Report Generator*, which uses price traces from [14] to produce accurate execution cost reports. In our evaluation, the Control Plane was configured to sample lifetimes between P10 and P90 and to apply a 100× time compression factor, allowing the full range of the lifetime distribution to be exercised within the evaluation window.

The Kaplan-Meier estimator [12] is a non-parametric survival model that estimates the probability of survival of a population over time. The Kaplan-Meier estimator runs in the background alongside the Spot Emulator, continuously estimating the survival probability of active spot VM instances based on their observed age and publishing per-instance survival probability signals to MongoDB. The Migrator and aggregators utilize them to proactively redirect workload away from instances at elevated risk of preemption and checkpoint in-memory model state before preemption occurs, as described in Section ?? and Section 5.2.

The emulator is deployed on a physical server that is logically connected to virtual machines through Kubernetes. The physical machine serves as the control plane and the worker nodes as servers in a datacenter. The pods deployed on the workers nodes are treated by the emulator as Spot VMs and are terminated based on their assigned lifetime.

6 Evaluation Setup

SpotLight was evaluated on Chameleon Cloud [13], a platform designed for experimental systems research. SpotLight was deployed on a five-node Kubernetes cluster, with node specifications listed in Table 1. MongoDB was hosted on a dedicated machine outside the cluster, with equivalent specifications to the host node. The evaluation focuses on two key aspects of the system: scalability and fault tolerance.

Table 1: Cluster Node Specifications

Specification	Host	Worker
CPU Model	Intel Xeon Gold 6126 @ 2.60GHz	—
vCPUs	48	16
Total RAM	187 GiB	28 GiB

6.1 Scalability

The scalability of SpotLight is evaluated across four metrics that capture the impact of increasing client load on system performance. SpotLight is evaluated against Papaya [8], a production asynchronous FL system, as the baseline, chosen for its support of asynchronous aggregation at scale. The workload used for the evaluation represents a cyclical pattern of client participation, reflecting natural increases and decreases in active client populations throughout the day, as shown in Figure 9. The evaluation uses a CNN model

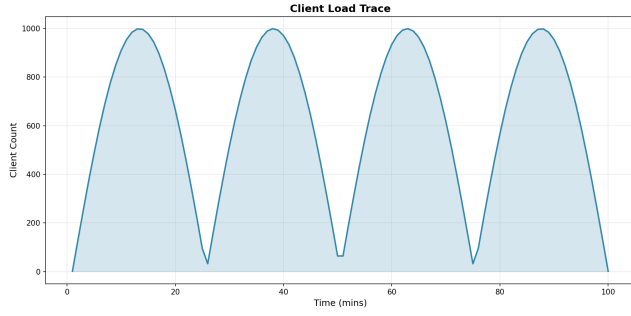


Figure 9: Cyclical workload pattern used in the scalability evaluation, reflecting natural increases and decreases in active client participation throughout the day.

with a 10MB payload to measure system scalability. The metrics used are as follows.

1) Client Utilization. Client utilization is defined as $\frac{T_{train}}{T_{train} + T_{aggregation} + T_{queue}}$,

where T_{train} denotes local computation at the client, $T_{aggregation}$ denotes the time spent waiting for aggregation to complete, and T_{queue} denotes the time buffered at the Sub-Global Aggregator prior to aggregation. Higher utilization indicates a greater fraction of total time spent on useful local computation rather than coordination overhead.

2) Aggregator Scaling. This metric evaluates how the aggregation tier scales relative to client load across all supported algorithms. A scalable design should demonstrate proportional and controlled growth in Sub-Global Aggregator instances as client population increases.

3) Queue Time. Queue time measures the duration each client spends waiting before its update enters aggregation, capturing contention at the aggregation layer. In a well-scaled system, queue time should remain bounded as client count grows.

4) Cost. Cost measures the total infrastructure expenditure required to operate SpotLight under varying client loads, expressed as instance-hours consumed by the aggregation tier over the duration of training.

Since Papaya does not support elastic scaling in response to client load, two configurations were derived to enable a fair comparison. SpotLight was first run on the workload shown in Figure 9, and its execution cost and average client utilization were recorded. These two parameters were used to derive the following Papaya configurations:

Papaya-Budget deploys a fixed number of aggregators at the same cost as SpotLight, measuring the client utilization and queue time achievable within an equivalent budget.

Papaya-Max determines the additional expenditure required for Papaya to match SpotLight’s client utilization and queue time, quantifying the cost overhead of operating without elastic scaling.

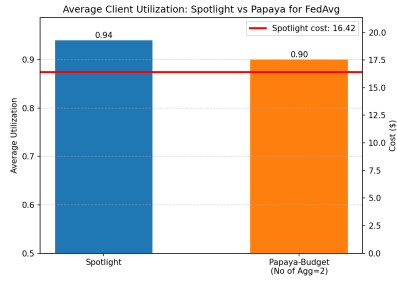
The evaluation were conducted for all algorithms listed in Table ?? . The results for SpotLight vs Papaya-Budget are shown in Figure 10.

7 Discussion

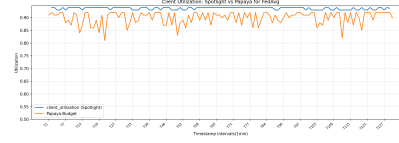
8 Related Work

9 Conclusion

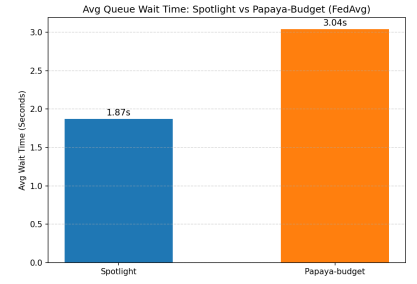
We present SpotLight...



(a) Average client utilization: SpotLight achieves 0.94 vs Papaya-Budget at 0.90 within the same cost budget (\$16.42).



(b) Client utilization over time: SpotLight maintains stable utilization while Papaya-Budget exhibits variance due to fixed aggregator capacity.



(c) Average queue wait time: SpotLight reduces wait time to 1.87s compared to 3.04s for Papaya-Budget, a 38.5% reduction.

Figure 10: Papaya-Budget comparison for FedAvg: SpotLight achieves higher client utilization and lower queue wait time than Papaya-Budget at equivalent cost.

References

- [1] Amazon Web Services. 2024. EC2 Spot Instance Interruption Notices. <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-instance-termination-notices.html>. Accessed: 2024.
- [2] Daniel J. Beutel, Taner Topal, Akhil Mathur, Xinchu Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Kwing Hei Li, Titouan Parcollet, Pedro Porto Buarque de Gusmão, and Nicholas D. Lane. 2022. Flower: A Friendly Federated Learning Research Framework. *arXiv preprint arXiv:2007.14390* (2022).
- [3] Kallista A. Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingberman, Vladimir Ivanov, Chloé Kiddon, Jakub Konečný, Stefano Mazzocchi, H. Brendan McMahan, Timon Van Overveldt, David Petrou, Daniel Ramage, and Jason Roselander. 2019. Towards Federated Learning at Scale: System Design. In *SysML*.
- [4] Jiangfei Duan, Ziang Song, Xupeng Miao, Xiaoli Xi, Dahua Lin, Harry Xu, Minjia Zhang, and Zhihao Jia. 2024. Parcae: proactive, liveput-optimized DNN training on preemptible instances. In *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation* (Santa Clara, CA, USA) (NSDI'24). USENIX Association, USA, Article 62, 19 pages.
- [5] Kazim Ergun, Rishikanth Chandrasekaran, and Tajana Rosing. 2023. Federated Hyperdimensional Computing. *arXiv preprint arXiv:2312.15966* (2023).
- [6] Google Cloud. 2024. Spot VMs. <https://cloud.google.com/compute/docs/instances/spot>. Accessed: 2024.
- [7] Jianwei Hao, Ting Jiang, Wei Wang, and In Kee Kim. 2021. An Empirical Analysis of VM Startup Times in Public IaaS Clouds. In *2021 IEEE 14th International Conference on Cloud Computing (CLOUD)*. 398–403. doi:10.1109/CLOUD53861.2021.00053
- [8] Dzmitry Huba, John Nguyen, Kshitiz Malik, Ruiyu Zhu, Mike Rabbat, Ashkan Yousefpour, Carole-Jean Wu, Hongyuan Zhan, Pavel Ustinov, Harish Srinivas, Kaikai Wang, Anthony Shoumikhin, Jesik Min, and Mani Malek. 2022. PAPAYA: Practical, Private, and Scalable Federated Learning. In *Proceedings of the 5th Conference on Machine Learning and Systems (MLSys 2022)*. https://proceedings.mlsys.org/paper_files/paper/2022/hash/a8bc4cb14a20f20d1f96188bd61eec87-Abstract.html
- [9] David Irwin, Prashant J. Shenoy, Pradeep Ambati, Prateek Sharma, Supreeth Shastri, and Ahmed Ali-Eldin. 2019. The Price Is (Not) Right: Reflections on Pricing for Transient Cloud Servers. In *28th International Conference on Computer Communication and Networks (ICCCN)*. 1–9.
- [10] K. R. Jayaram, Vinod Muthusamy, Gegi Thomas, Ashish Verma, and Mark Purcell. 2022. Adaptive Aggregation for Federated Learning. *arXiv preprint arXiv:2203.12163* (2022).
- [11] Pedro Joaquim, Manuel Bravo, Luis Rodrigues, and Miguel Matos. 2019. Hourglass: Leveraging Transient Resources for Time-Constrained Graph Processing in the Cloud. In *The Fourteenth EuroSys Conference (EuroSys)*.
- [12] E. L. Kaplan and Paul Meier. 1958. Nonparametric Estimation from Incomplete Observations. *J. Amer. Statist. Assoc.* 53, 282 (1958), 457–481. doi:10.1080/01621459.1958.10501452
- [13] Kate Keahey, Jason Anderson, Zhuo Zhen, Pierre Riteau, Paul Ruth, Dan Stanzione, Mert Cevik, Jacob Collieran, Haryadi S. Gunawi, Cody Hammock, Joe Mambretti, Alexander Barnes, François Halbach, Alex Rocha, and Joe Stubbs. 2020. Lessons Learned from the Chameleon Testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association.
- [14] Sungjae Lee, Jaeh Hwang, and Kyungyong Lee. 2022. SpotLake: Diverse Spot Instance Dataset Archive Service. In *2022 IEEE International Symposium on Workload Characterization (IISWC)*. 242–255. doi:10.1109/IISWC55918.2022.00029
- [15] Mu Li, David G. Andersen, Jun Woo Park, Alexander J. Smola, Amr Ahmed, Vanja Josifovski, James Long, Eugene J. Shekita, and Bor-Yiing Su. 2014. Scaling Distributed Machine Learning with the Parameter Server. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*. USENIX Association, Broomfield, CO, 583–598.
- [16] Zhifei Li, Tian Xia, Ziming Mao, Zihan Zhou, Ethan J. Jackson, Jamison Kerney, Zhanghao Wu, Pratik Mishra, Yi Xu, Yifan Qiao, Scott Shenker, and Ion Stoica. 2026. SkyNomad: On Using Multi-Region Spot Instances to Minimize AI Batch Job Cost. CoRR abs/2601.06520 (2026). arXiv:2601.06520 doi:10.48550/arXiv.2601.06520
- [17] Yang Liu, Tao Fan, Tianjian Chen, Qian Xu, and Qiang Yang. 2021. FATE: An Industrial Grade Platform for Collaborative Learning with Data Protection. *Journal of Machine Learning Research* 22, 226 (2021), 1–6.
- [18] Heiko Ludwig, Nathalie Baracaldo, Gegi Thomas, Yi Zhou, Ali Anwar, Shashank Rajamoni, Yuya Ong, Jayaram Radhakrishnan, Ashish Verma, Mathieu Sinn, et al. 2020. IBM Federated Learning: An Enterprise Framework White Paper V0.1. *arXiv preprint arXiv:2007.10987* (2020).
- [19] H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*. 1273–1282.
- [20] John Nguyen, Kshitiz Malik, Hongyuan Zhan, Ashkan Yousefpour, Mike Rabbat, Mani Malek, and Dzmitry Huba. 2022. Federated Learning with Buffered Asynchronous Aggregation. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 3581–3607.
- [21] NVIDIA. 2025. Efficient Federated Learning in the Era of LLMs with Message Quantization and Streaming. NVIDIA Technical Blog. <https://developer.nvidia.com/blog/efficient-federated-learning-in-the-era-of-llms-with-message-quantization-and-streaming/>
- [22] Shixiong Qi, K. K. Ramakrishnan, and Myungjin Lee. 2024. LiFL: A Lightweight, Event-driven Serverless Platform for Federated Learning. In *Proceedings of Machine Learning and Systems (MLSys)*.
- [23] Sashank Reddi, Zachary Charles, Manzil Zaheer, Zachary Garrett, Keith Rush, Jakub Konečný, Sanjiv Kumar, and H. Brendan McMahan. 2021. Adaptive Federated Optimization. In *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2003.00295>
- [24] Ruitao Shang, Fei Xu, Zhuoyan Bai, Li Chen, Zhi Zhou, and Fangming Liu. 2023. spotDNN: Provisioning Spot Instances for Predictable Distributed DNN Training in the Cloud. In *2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS)*. 1–10.
- [25] Prateek Sharma, JCS Kadupitiya, and Vikram Jadhao. 2019. Modeling Constrained Preemption Dynamics Of Transient Cloud Servers. *arXiv* (2019), arXiv–1911.
- [26] Supreeth Subramanya, Amr Rizk, and David Irwin. 2016. Cloud Spot Markets are Not Sustainable: The Case for Transient Guarantees. In *8th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud)*.
- [27] Yan Sun, Li Shen, Hao Sun, Liang Ding, and Dacheng Tao. 2023. Efficient Federated Learning Via Local Adaptive Amended Optimizer With Linear Speedup. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 12 (2023), 14453–14464. doi:10.1109/TPAMI.2023.3300886
- [28] John Thorpe, Pengzhan Zhao, Jonathan Eyolfson, Yifan Qiao, Zhihao Jia, Minjia Zhang, Ravi Netravali, and Guoqing Harry Xu. 2023. Bamboo: Making Preemptible Instances Resilient for Affordable Training of Large DNNs. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 497–513.
- [29] Zhanghao Wu et al. 2024. SkyServe: Serving AI Models across Regions and Clouds with Spot Instances. *arXiv preprint arXiv:2411.01438* (2024).
- [30] Zhanghao Wu, Wei-Lin Chiang, Ziming Mao, Zongheng Yang, Eric Friedman, Scott Shenker, and Ion Stoica. 2024. Can't Be Late: Optimizing Spot Instance Savings under Deadlines. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 185–203.
- [31] Cong Xie, Sanmi Koyejo, and Indranil Gupta. 2019. Asynchronous Federated Optimization. *arXiv preprint arXiv:1903.03934* (2019).
- [32] Fangkai Yang, Lu Wang, Zhenyu Xu, Jue Zhang, Liqun Li, Bo Qiao, Camille Couturier, Chetan Bansal, Soumya Ram, Si Qin, Zhen Ma, Íñigo Goiri, Eli Cortez, Terry Yang, Victor Rühle, Saravan Rajmohan, Qingwei Lin, and Dongmei Zhang. 2023. Snape: Reliable and Low-Cost Computing with Mixture of Spot and On-Demand VMs. In *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.